```
// -------- src/main.ts --------
// 入口
import { Game } from './Game';
const game = new Game();
game.start().catch((e) => {
  console.error('[main] 启动失败:', e);
});
// -------- src/Game.ts --------
// 游戏组合根:装配所有系统与场景,接管生命周期
import { Renderer } from './core/Renderer';
import { GameLoop } from './core/GameLoop';
import { SceneManager } from './core/Scene';
import { Ticker } from './core/Ticker';
import { AudioMgr } from './core/AudioMgr';
import { SaveManager, Storage } from './core/Storage';
import { Toast } from './ui/Toast';
import { bus, Events } from './core/EventBus';
import { SaveData } from './data/types';
import { FEEL } from './data/configs';
import { MeritSystem } from './systems/MeritSystem';
import { ComboSystem } from './systems/ComboSystem';
import { LevelSystem } from './systems/LevelSystem';
import { DailySystem } from './systems/DailySystem';
import { SkinSystem } from './systems/SkinSystem';
import { SyncService } from './systems/SyncService';
import { ShareSystem } from './systems/ShareSystem';
import { AdSystem } from './systems/AdSystem';
import { Analytics } from './systems/Analytics';
import { GongfaSystem } from './systems/GongfaSystem';
import { EventSystem } from './systems/EventSystem';
import { RebirthSystem } from './systems/RebirthSystem';
import { VERSES } from './data/strings';
import { ringPitch } from './utils/practiceRules';
import { BootScene } from './scenes/BootScene';
import { HomeScene } from './scenes/HomeScene';
import { CultivateScene } from './scenes/CultivateScene';
import { DailyScene } from './scenes/DailyScene';
import { RankScene } from './scenes/RankScene';
import { SettingScene } from './scenes/SettingScene';
export class Game {
  renderer = new Renderer();
  loop = new GameLoop();
  scenes = new SceneManager();
  ticker = new Ticker();
  audio = new AudioMgr();
  toast = new Toast();
  saveManager!: SaveManager;
  // 系统
  merit!: MeritSystem;
  combo!: ComboSystem;
```

```
  levelSystem!: LevelSystem;
  daily!: DailySystem;
  skin!: SkinSystem;
  sync!: SyncService;
  share!: ShareSystem;
  ad!: AdSystem;
  analytics!: Analytics;
  gongfa!: GongfaSystem;
  events!: EventSystem;
  rebirth!: RebirthSystem;
  /** 当日偈语(启动时抽取,跨场景一致) */
  dailyVerse = '';
  readonly bootAt = Date.now();
  private firstFrameRendered = false;
  get save(): SaveData {
    return this.saveManager.save;
  }
  async start(): Promise<void> {
    // 1. 渲染器
    this.renderer.init();
    this.loop.lowFpsMode = this.renderer.lowEnd;
    // 2. 存档
    this.saveManager = new SaveManager(Storage.load());
    // 3. 音频(需等待首次触摸激活)
    this.audio.init();
    this.audio.setSoundOn(this.save.soundOn);
    this.audio.setTapSound(
      this.save.tapSound === 'wooden' ? 'wooden'
        : this.save.tapSound === 'crisp' ? 'crisp'
        : this.save.tapSound === 'thump' ? 'thump'
        : 'resonant'
    );
    this.audio.setAmbience(!!this.save.extra.rainOn, !!this.save.extra.birdOn);
    // 4. 系统装配
    this.combo = new ComboSystem();
    this.levelSystem = new LevelSystem(this);
    this.merit = new MeritSystem(this);
    this.daily = new DailySystem(this);
    this.skin = new SkinSystem(this);
    this.sync = new SyncService(this);
    this.share = new ShareSystem(this);
    this.ad = new AdSystem(this);
    this.analytics = new Analytics(this);
    this.gongfa = new GongfaSystem(this);
    this.events = new EventSystem(this);
    this.rebirth = new RebirthSystem(this);
    this.analytics.init();
    this.levelSystem.checkLevelUp();
    this.gongfa.refresh();
    // 5. 每日重置 + 偈语
```

```
    this.daily.rolloverIfNeeded();
    this.dailyVerse = VERSES[new Date().getDate() % VERSES.length];
    // 6. 输入
    this.bindInput();
    // 7. 生命周期
    this.bindLifecycle();
    // 8. 云登录(异步,不阻塞启动)
    this.sync.init();
    // 9. 进入启动场景
    this.scenes.replace(new BootScene(this));
    // 10. 主循环
    this.loop.start((dt) => this.frame(dt));
    this.analytics.track('launch', {
      level: this.levelSystem.current.name,
      merit: Math.floor(this.save.merit),
    });
  }
  private frame(dt: number): void {
    this.audio.setRingScale(ringPitch(this.save.totalTaps));
    this.ticker.update(dt);
    this.combo.update(dt);
    this.events.update(dt);
    this.saveManager.tick();
    this.scenes.update(dt);
    this.renderer.clear();
    this.scenes.render(this.renderer.ctx);
    this.toast.render(this.renderer.ctx, this.renderer.width, this.renderer.height);
    if (!this.firstFrameRendered) {
      this.firstFrameRendered = true;
      console.log('[Game] first frame rendered');
    }
  }
  // ---------- 输入:单指敲击 + 触摸合并(80ms 防抖) ----------
  private lastTapAt = 0;
  private activeTouchId: number | null = null;
  private bindInput(): void {
    wx.onTouchStart((e: any) => {
      const t = e.touches[0];
      if (!t) return;
      // 多点触控:只跟踪第一根手指
      if (this.activeTouchId !== null) return;
      this.activeTouchId = t.identifier ?? 0;
      this.audio.unlock(); // iOS 激活音频
      const now = Date.now();
      const x = t.clientX;
      const y = t.clientY;
      // 主界面防抖动。听经要精确连点,不走这道合并。
      if (!this.scenes.top?.preciseTouch && now - this.lastTapAt < FEEL.tapMergeMs) return;
      this.lastTapAt = now;
      this.scenes.top?.onTouchStart(x, y);
```

```
    });
    wx.onTouchMove((e: any) => {
      const t = e.touches[0];
      if (!t) return;
      this.scenes.top?.onTouchMove(t.clientX, t.clientY);
    });
    const onEnd = (e: any) => {
      const t = (e.changedTouches && e.changedTouches[0]) || null;
      this.activeTouchId = null;
      if (t) this.scenes.top?.onTouchEnd(t.clientX, t.clientY);
    };
    wx.onTouchEnd(onEnd);
    wx.onTouchCancel(onEnd);
  }
  // ---------- 生命周期 ----------
  private bindLifecycle(): void {
    wx.onHide(() => {
      this.loop.pause();
      this.audio.pauseBgm();
      this.audio.pauseAmbience();
      this.saveManager.flush();          // 强写本地
      this.analytics.sessionEnd();       // 结束会话埋点
      this.analytics.destroy();          // 清理定时器
      this.sync.flush(true);             // 尽力同步,并刷新离开时间
    });
    wx.onShow(() => {
      this.loop.resume();
      this.audio.resumeBgm();
      this.audio.resumeAmbience();
      this.analytics.init();             // 重建埋点定时器
      this.daily.rolloverIfNeeded();
      // 回到前台检查离线收益(罗汉解锁后)
      bus.emit(Events.OFFLINE_REWARD, { check: true });
    });
    // 屏幕常亮(修行时)
    try {
      wx.setKeepScreenOn({ keepScreenOn: true });
    } catch (e) {
      /* ignore */
    }
  }
}
// -------- src/core/Renderer.ts --------
// 渲染器:Canvas 封装、DPR 高清适配、安全区
export class Renderer {
  canvas: any = null;
  ctx!: CanvasRenderingContext2D;
  /** 逻辑像素尺寸(UI 坐标系) */
  width = 0;
  height = 0;
```

```
  dpr = 1;
  safeTop = 0;
  safeBottom = 0;
  /** 低端机标记:降低粒子与帧率 */
  lowEnd = false;
  init(): void {
    this.canvas = wx.createCanvas();
    this.ctx = this.canvas.getContext('2d') as CanvasRenderingContext2D;
    const info = wx.getWindowInfo();
    this.dpr = info.pixelRatio || 1;
    this.width = info.windowWidth;
    this.height = info.windowHeight;
    this.canvas.width = Math.floor(this.width * this.dpr);
    this.canvas.height = Math.floor(this.height * this.dpr);
    this.ctx.scale(this.dpr, this.dpr);
    const safe = info.safeArea;
    if (safe) {
      this.safeTop = safe.top;
      this.safeBottom = Math.max(0, info.windowHeight - safe.bottom);
    }
    // 低端机探测:屏幕小或 DPR 低的设备降配
    this.lowEnd = this.width * this.height * this.dpr < 750 * 1334 * 1.5;
  }
  clear(): void {
    this.ctx.clearRect(0, 0, this.width, this.height);
  }
  /** 内容安全区域(避开刘海/Home 指示条) */
  get contentTop(): number {
    return this.safeTop + 8;
  }
  get contentBottom(): number {
    return this.height - this.safeBottom - 8;
  }
  get cx(): number {
    return this.width / 2;
  }
}
// -------- src/core/GameLoop.ts --------
// 主循环:RAF 驱动、切后台停帧、低端机降频
export type FrameCallback = (dtMs: number) => void;
export class GameLoop {
  private running = false;
  private lastTime = 0;
  private cb: FrameCallback | null = null;
  private accumulator = 0;
  /** 低端机 30fps 运行 */
  lowFpsMode = false;
  start(cb: FrameCallback): void {
    this.cb = cb;
    this.running = true;
```

```
    this.lastTime = Date.now();
    requestAnimationFrame(this.tick);
  }
  private tick = (): void => {
    if (!this.running) return;
    const now = Date.now();
    let dt = now - this.lastTime;
    this.lastTime = now;
    if (dt > 100) dt = 100; // 切后台回来避免大步长跳变
    if (this.lowFpsMode) {
      // 30fps:攒够 33ms 才跑一帧;dt 钳制到 100ms 避免动画跳变
      this.accumulator += dt;
      if (this.accumulator >= 33) {
        this.cb?.(Math.min(this.accumulator, 100));
        this.accumulator = 0;
      }
    } else {
      this.cb?.(dt);
    }
    requestAnimationFrame(this.tick);
  };
  pause(): void {
    this.running = false;
  }
  resume(): void {
    if (this.running) return;
    this.running = true;
    this.lastTime = Date.now();
    requestAnimationFrame(this.tick);
  }
}
// -------- src/core/Ticker.ts --------
// 缓动函数 + 补间动画驱动
export const Easing = {
  linear: (t: number) => t,
  easeOutQuad: (t: number) => t * (2 - t),
  easeInQuad: (t: number) => t * t,
  easeOutCubic: (t: number) => 1 - Math.pow(1 - t, 3),
  easeOutBack: (t: number) => {
    const c = 1.70158;
    return 1 + (c + 1) * Math.pow(t - 1, 3) + c * Math.pow(t - 1, 2);
  },
  easeInOutSine: (t: number) => -(Math.cos(Math.PI * t) - 1) / 2,
};
interface Tween {
  duration: number;
  elapsed: number;
  update: (t: number) => void;
  complete?: () => void;
  easing: (t: number) => number;
```

```
  done: boolean;
}
interface Delayed {
  at: number;
  cb: () => void;
  done: boolean;
}
// 每帧由 GameLoop 驱动;管理所有 UI 动画
export class Ticker {
  private tweens: Tween[] = [];
  private delayed: Delayed[] = [];
  private now = 0;
  /** duration 毫秒内,update 收到缓动后的 t(0→1) */
  to(
    duration: number,
    update: (t: number) => void,
    opts?: { easing?: (t: number) => number; complete?: () => void }
  ): void {
    this.tweens.push({
      duration: Math.max(1, duration),
      elapsed: 0,
      update,
      complete: opts?.complete,
      easing: opts?.easing ?? Easing.easeOutQuad,
      done: false,
    });
  }
  /** 延迟执行(毫秒) */
  delay(ms: number, cb: () => void): void {
    this.delayed.push({ at: this.now + ms, cb, done: false });
  }
  update(dtMs: number): void {
    this.now += dtMs;
    for (const tw of this.tweens) {
      if (tw.done) continue;
      tw.elapsed += dtMs;
      const t = Math.min(1, tw.elapsed / tw.duration);
      tw.update(tw.easing(t));
      if (t >= 1) {
        tw.done = true;
        try {
          tw.complete?.();
        } catch (e) {
          console.error('[Ticker] complete error:', e);
        }
      }
    }
    if (this.tweens.length > 64 || this.tweens.some((t) => t.done)) {
      this.tweens = this.tweens.filter((t) => !t.done);
    }
```

```
    for (const d of this.delayed) {
      if (!d.done && this.now >= d.at) {
        d.done = true;
        try {
          d.cb();
        } catch (e) {
          console.error('[Ticker] delay cb error:', e);
        }
      }
    }
    this.delayed = this.delayed.filter((d) => !d.done);
  }
}
// -------- src/core/Scene.ts --------
// 场景基类 + 栈式场景管理
// 约定:全屏场景用 replace 切换;弹层面板用 push/pop(仅栈顶接收输入,渲染只画栈顶)
import type { Game } from '../Game';
export abstract class Scene {
  protected game: Game;
  /** 听经等需要精确连点的场景跳过主界面 80ms 防抖 */
  preciseTouch = false;
  constructor(game: Game) {
    this.game = game;
  }
  /** 进入场景(push/replace 后调用) */
  abstract enter(params?: any): void;
  /** 离开场景(被 pop/replace 时调用) */
  abstract exit(): void;
  abstract update(dtMs: number): void;
  abstract render(ctx: CanvasRenderingContext2D): void;
  /** 返回 true 表示触摸已被消费 */
  onTouchStart(_x: number, _y: number): boolean {
    return false;
  }
  onTouchMove(_x: number, _y: number): void {}
  onTouchEnd(_x: number, _y: number): void {}
}
export class SceneManager {
  private stack: Scene[] = [];
  get top(): Scene | null {
    return this.stack.length ? this.stack[this.stack.length - 1] : null;
  }
  replace(scene: Scene, params?: any): void {
    while (this.stack.length) this.stack.pop()!.exit();
    this.stack.push(scene);
    scene.enter(params);
  }
  push(scene: Scene, params?: any): void {
    this.stack.push(scene);
    scene.enter(params);
```

```
  }
  pop(): void {
    const s = this.stack.pop();
    s?.exit();
  }
  update(dtMs: number): void {
    this.top?.update(dtMs);
  }
  render(ctx: CanvasRenderingContext2D): void {
    this.top?.render(ctx);
  }
}
// -------- src/core/EventBus.ts --------
// 全局事件总线:模块间解耦通信
export type EventHandler = (payload?: any) => void;
export class EventBus {
  private handlers: Map<string, Set<EventHandler>> = new Map();
  on(event: string, handler: EventHandler): () => void {
    if (!this.handlers.has(event)) this.handlers.set(event, new Set());
    this.handlers.get(event)!.add(handler);
    return () => this.off(event, handler);
  }
  off(event: string, handler: EventHandler): void {
    this.handlers.get(event)?.delete(handler);
  }
  emit(event: string, payload?: any): void {
    const set = this.handlers.get(event);
    if (!set) return;
    set.forEach((h) => {
      try {
        h(payload);
      } catch (e) {
        console.error(`[EventBus] handler error on "${event}":`, e);
      }
    });
  }
  clear(event?: string): void {
    if (event) this.handlers.delete(event);
    else this.handlers.clear();
  }
}
export const bus = new EventBus();
// 事件名常量,避免字符串散落
export const Events = {
  MERIT_CHANGED: 'merit:changed',       // { merit, gain }
  COMBO_CHANGED: 'combo:changed',       // { combo }
  COMBO_MILESTONE: 'combo:milestone',   // { combo } 10/50/100
  LEVEL_UP: 'level:up',                 // { levelIndex, name }
  DAILY_GOAL: 'daily:goal',             // 今日功课敲满 100
  DAILY_CLAIMED: 'daily:claimed',       // { reward, streak }
```

```
  SKIN_CHANGED: 'skin:changed',
  SCENE_CHANGED: 'scene:changed',
  SYNC_STATE: 'sync:state',             // 'online' | 'guest' | 'syncing' | 'error'
  OFFLINE_REWARD: 'offline:reward',     // { merit, hours }
  SAVE_DIRTY: 'save:dirty',
} as const;
// -------- src/core/Storage.ts --------
// 本地存档:全量 JSON 镜像 + 写入节流
import { SaveData, createDefaultSave, migrateSave } from '../data/types';
const SAVE_KEY = 'kxcf_save_v1';
export class Storage {
  /** 读取本地存档(无则返回默认) */
  static load(): SaveData {
    try {
      const raw = wx.getStorageSync(SAVE_KEY);
      if (!raw) return createDefaultSave();
      return migrateSave(JSON.parse(raw as string));
    } catch (e) {
      console.warn('[Storage] load failed, use default:', e);
      return createDefaultSave();
    }
  }
  static write(data: SaveData): void {
    try {
      wx.setStorageSync(SAVE_KEY, JSON.stringify(data));
    } catch (e) {
      console.warn('[Storage] write failed:', e);
    }
  }
  static clear(): void {
    try {
      wx.removeStorageSync(SAVE_KEY);
    } catch (e) {
      /* ignore */
    }
  }
}
/** 写入节流:≥1s 或关键节点强写 */
export class SaveManager {
  private data: SaveData;
  private dirty = false;
  private lastWrite = 0;
  constructor(data: SaveData) {
    this.data = data;
  }
  get save(): SaveData {
    return this.data;
  }
  /** 标记脏,由 tick 节流落盘 */
  markDirty(): void {
```

```
    this.dirty = true;
  }
  tick(): void {
    if (!this.dirty) return;
    const now = Date.now();
    if (now - this.lastWrite < 1000) return;
    this.flush();
  }
  /** 立即落盘(切后台/关键节点调用) */
  flush(): void {
    this.data.lastSeenAt = Date.now();
    Storage.write(this.data);
    this.dirty = false;
    this.lastWrite = Date.now();
  }
  hasPending(): boolean {
    return this.dirty;
  }
}
// -------- src/core/AudioMgr.ts --------
// 音频系统:敲击音 WebAudio 合成(零音频文件)+ BGM 流式播放
// iOS 注意:AudioContext 需首次 touchstart 后 resume 才可出声(首个点击前静默属正常)
import type { SkinConfig, BgmConfig } from '../data/configs';
import { BGMS } from '../data/configs';
export type TapSoundStyle = 'resonant' | 'wooden' | 'crisp' | 'thump';
export class AudioMgr {
  private ctx: any = null;
  private unlocked = false;
  private soundOn = true;
  private tapSound: TapSoundStyle = 'resonant';
  /** 年轮把基频乘到 0.90–1,音色档位本身不变 */
  private ringScale = 1;
  private bgm: any = null; // InnerAudioContext 单例
  private bgmId = 'none';
  private bgmTargetVolume = 0.6;
  private fadeTimer: any = null;
  private rainOn = false;
  private birdOn = false;
  private rainSrc: any = null;
  private birdSrc: any = null;
  private ambiencePaused = false;
  init(): void {
    try {
      this.ctx = wx.createWebAudioContext();
    } catch (e) {
      console.warn('[Audio] WebAudio unavailable:', e);
      this.ctx = null;
    }
  }
  setSoundOn(on: boolean): void {
```

```
    this.soundOn = on;
    if (!on) {
      this.destroyBgm();
      this.stopAmbienceLoops();
      return;
    }
    if (this.bgmId !== 'none') {
      const id = this.bgmId;
      this.bgmId = '';
      this.playBgm(id);
    }
    this.syncAmbience();
  }
  /** 切换木鱼音色:'resonant'(腔体共振)| 'wooden'(木质敲击)| 'crisp'(清脆实木)| 'thump'(咚咚木鱼) */
  setTapSound(style: TapSoundStyle): void {
    this.tapSound = style;
  }
  setRingScale(scale: number): void {
    this.ringScale = scale > 0 ? scale : 1;
  }
  /** 必须在首次 touchstart 中调用(iOS 激活策略) */
  unlock(): void {
    if (this.unlocked) return;
    this.unlocked = true;
    try {
      if (this.ctx && this.ctx.state === 'suspended') this.ctx.resume();
      this.syncAmbience();
    } catch (e) {
      /* ignore */
    }
  }
  // ---------- 敲击音:真实木鱼合成 ----------
  // 四种音色:
  //   resonant = 腔体共振(基频+非整数比泛音,圆润有共鸣)
  //   wooden   = 木质敲击(低基频三角波+强瞬态,短促干脆的木质感)
  //   crisp    = 清脆实木(极高频瞬态+多层高频泛音,明亮干脆带"啪"感)
  //   thump    = 咚咚木鱼(低频腔体+中低频泛音,去金属感,浑厚木鱼"咚")
  playTap(skin: SkinConfig): void {
    if (this.tapSound === 'wooden') this.playTapWooden(skin);
    else if (this.tapSound === 'crisp') this.playTapCrisp(skin);
    else if (this.tapSound === 'thump') this.playTapThump(skin);
    else this.playTapResonant(skin);
  }
  /** 腔体共振音色(默认):槌击噪声瞬态 + 多泛音腔体共振 */
  private playTapResonant(skin: SkinConfig): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      // 基频 ±2.5% 随机,听感更自然
      const jitter = 1 + (Math.random() * 0.05 - 0.025);
```

```
      const f = skin.freq * jitter * this.ringScale;
      const d = skin.decay;
      // 1) 槌击瞬态:短噪声(木质"哒"的接触声)
      this.tapThump(t, 0.22, 0.018, 1000);
      // 2) 腔体共振:基频 + 非整数比泛音(木质腔体特征,泛音越高衰减越快)
      this.tapTone(f, d, 0.9, t, 'sine');         // 基频
      this.tapTone(f * 2.01, d * 0.7, 0.32, t, 'sine'); // ~2 倍(木质非严格整数比)
      this.tapTone(f * 2.93, d * 0.5, 0.16, t, 'sine'); // ~3 倍
      this.tapTone(f * 4.12, d * 0.35, 0.07, t, 'sine');// ~4 倍
      this.tapTone(f * 5.8, d * 0.22, 0.04, t, 'sine'); // 高频木质泛音(脆)
    } catch (e) {
      /* 音频失败不影响游戏 */
    }
  }
  /** 木质敲击音色:低基频三角波 + 强瞬态,更干更脆的木鱼敲击感 */
  private playTapWooden(skin: SkinConfig): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      const jitter = 1 + (Math.random() * 0.06 - 0.03);
      // 木质音色基频更低(≈0.72×),声音更"实"
      const f = skin.freq * 0.72 * jitter * this.ringScale;
      const d = skin.decay * 0.9;
      // 1) 强槌击瞬态(木质敲击的接触冲击更明显)
      this.tapThump(t, 0.32, 0.014, 1400);
      // 2) 木质腔体:三角波基频(更丰富的低次谐波,木质味)+ 少量泛音
      this.tapTone(f, d, 0.95, t, 'triangle');
      this.tapTone(f * 2.02, d * 0.6, 0.24, t, 'sine');
      this.tapTone(f * 3.05, d * 0.38, 0.1, t, 'sine');
    } catch (e) {
      /* 音频失败不影响游戏 */
    }
  }
  /** 清脆实木音色:极高频瞬态 + 多层高频泛音,明亮干脆带"啪"感(原始 D 版) */
  private playTapCrisp(skin: SkinConfig): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      const jitter = 1 + (Math.random() * 0.05 - 0.025);
      const f = skin.freq * jitter * this.ringScale;
      const d = skin.decay * 0.9;
      // 1) 极高频瞬态(硬木接触"啪")+ 中频冲击(带通"咔")
      this.tapThump(t, 0.34, 0.006, 2800);
      this.tapThumpBand(t, 0.18, 0.014, 1100);
      // 2) 低频体共振 + 基频 + 多层高频泛音(更亮)
      this.tapTone(f * 0.55, d * 0.4, 0.16, t, 'sine');
      this.tapTone(f, d, 0.82, t, 'sine');
      this.tapTone(f * 2.42, d * 0.5, 0.34, t, 'sine');
      this.tapTone(f * 3.85, d * 0.38, 0.18, t, 'sine');
      this.tapTone(f * 6.1, d * 0.26, 0.09, t, 'sine');
```

```
      this.tapTone(f * 8.4, d * 0.18, 0.04, t, 'sine');
    } catch (e) {
      /* 音频失败不影响游戏 */
    }
  }
  /** 咚咚木鱼音色:低频腔体 + 中低频泛音,去金属感的浑厚木鱼"咚" */
  private playTapThump(skin: SkinConfig): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      const jitter = 1 + (Math.random() * 0.05 - 0.025);
      const f = skin.freq * jitter * this.ringScale;
      const d = skin.decay * 1.15; // 余韵更长
      // 1) 瞬态:中频为主(木质"哒",避免过高频的金属"叮")
      this.tapThump(t, 0.28, 0.007, 2000);
      this.tapThumpBand(t, 0.22, 0.016, 900);
      // 2) 低频腔体"咚" + 基频(稍低) + 中低频泛音(去高频金属泛音)
      this.tapTone(f * 0.45, d * 0.7, 0.4, t, 'sine');   // 低频体共振(更厚更"咚")
      this.tapTone(f * 0.92, d, 0.85, t, 'sine');        // 基频稍低,更闷更木
      this.tapTone(f * 1.83, d * 0.6, 0.28, t, 'sine');  // 中频腔体共鸣
      this.tapTone(f * 2.7, d * 0.45, 0.12, t, 'sine');
      this.tapTone(f * 4.2, d * 0.3, 0.05, t, 'sine');   // 只留少量高频泛音
    } catch (e) {
      /* 音频失败不影响游戏 */
    }
  }
  /** 带通瞬态噪声(中频木质冲击"咔") */
  private tapThumpBand(startAt: number, vol: number, dur: number, center: number): void {
    try {
      const sr = this.ctx.sampleRate || 44100;
      const len = Math.floor(sr * dur);
      const buffer = this.ctx.createBuffer(1, len, sr);
      const data = buffer.getChannelData(0);
      for (let i = 0; i < len; i++) {
        data[i] = (Math.random() * 2 - 1) * (1 - i / len);
      }
      const noise = this.ctx.createBufferSource();
      noise.buffer = buffer;
      const bp = this.ctx.createBiquadFilter();
      bp.type = 'bandpass';
      bp.frequency.value = center;
      bp.Q.value = 1.2;
      const gain = this.ctx.createGain();
      gain.gain.setValueAtTime(vol, startAt);
      gain.gain.exponentialRampToValueAtTime(0.0001, startAt + dur);
      noise.connect(bp);
      bp.connect(gain);
      gain.connect(this.ctx.destination);
      noise.start(startAt);
      noise.stop(startAt + dur + 0.005);
```

```
    } catch (e) {
      /* 瞬态噪声失败不影响主音 */
    }
  }
  /** 腔体共振单音(指数衰减包络,attack 极短) */
  private tapTone(freq: number, decay: number, vol: number, startAt: number, wave: OscillatorType = 'sine'): v
oid {
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    osc.type = wave;
    osc.frequency.setValueAtTime(freq, startAt);
    gain.gain.setValueAtTime(0.0001, startAt);
    gain.gain.exponentialRampToValueAtTime(vol, startAt + 0.002); // 2ms 起音(木质硬击)
    gain.gain.exponentialRampToValueAtTime(0.0001, startAt + decay);
    osc.connect(gain);
    gain.connect(this.ctx.destination);
    osc.start(startAt);
    osc.stop(startAt + decay + 0.05);
  }
  /** 槌击瞬态噪声(高通滤波保留高频接触声) */
  private tapThump(startAt: number, vol: number, dur: number, hpFreq: number): void {
    try {
      const sr = this.ctx.sampleRate || 44100;
      const len = Math.floor(sr * dur);
      const buffer = this.ctx.createBuffer(1, len, sr);
      const data = buffer.getChannelData(0);
      for (let i = 0; i < len; i++) {
        data[i] = (Math.random() * 2 - 1) * (1 - i / len); // 白噪声,线性衰减
      }
      const noise = this.ctx.createBufferSource();
      noise.buffer = buffer;
      const hp = this.ctx.createBiquadFilter();
      hp.type = 'highpass';
      hp.frequency.value = hpFreq;
      const gain = this.ctx.createGain();
      gain.gain.setValueAtTime(vol, startAt);
      gain.gain.exponentialRampToValueAtTime(0.0001, startAt + dur);
      noise.connect(hp);
      hp.connect(gain);
      gain.connect(this.ctx.destination);
      noise.start(startAt);
      noise.stop(startAt + dur + 0.005);
    } catch (e) {
      /* 瞬态噪声失败不影响主音 */
    }
  }
  private tone(freq: number, decay: number, wave: OscillatorType, vol: number, startAt: number): void {
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    osc.type = wave;
```

```
    osc.frequency.setValueAtTime(freq, startAt);
    gain.gain.setValueAtTime(0.0001, startAt);
    gain.gain.exponentialRampToValueAtTime(vol, startAt + 0.006);
    gain.gain.exponentialRampToValueAtTime(0.0001, startAt + decay);
    osc.connect(gain);
    gain.connect(this.ctx.destination);
    osc.start(startAt);
    osc.stop(startAt + decay + 0.05);
  }
  /** 境界突破:上行琶音 */
  playBreakthrough(): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      const notes = [261.6, 329.6, 392.0, 523.3]; // C E G C
      notes.forEach((f, i) => this.tone(f, 0.6, 'sine', 0.4, t + i * 0.12));
    } catch (e) {
      /* ignore */
    }
  }
  /** 连击里程碑:清脆一声 */
  playComboChime(): void {
    if (!this.soundOn || !this.ctx || !this.unlocked) return;
    try {
      const t: number = this.ctx.currentTime;
      this.tone(880, 0.35, 'sine', 0.3, t);
      this.tone(1320, 0.25, 'sine', 0.15, t + 0.05);
    } catch (e) {
      /* ignore */
    }
  }
  // ---------- BGM ----------
  playBgm(id: string): void {
    if (id === this.bgmId && this.bgm) return;
    this.destroyBgm();
    this.bgmId = id;
    const cfg = BGMS.find((b: BgmConfig) => b.id === id);
    if (!cfg || !cfg.url || !this.soundOn || id === 'none') return;
    try {
      this.bgm = wx.createInnerAudioContext();
      this.bgm.src = cfg.url;
      this.bgm.loop = true;
      this.bgm.volume = 0;
      this.bgm.play();
      this.fadeTo(this.bgmTargetVolume, 500);
      this.bgm.onError((e: any) => {
        console.warn('[Audio] BGM error:', e);
      });
    } catch (e) {
      console.warn('[Audio] BGM play failed:', e);
```

```
      this.bgm = null;
    }
  }
  stopBgm(): void {
    this.destroyBgm();
    this.bgmId = 'none';
  }
  private destroyBgm(): void {
    if (this.fadeTimer) {
      clearInterval(this.fadeTimer);
      this.fadeTimer = null;
    }
    if (!this.bgm) return;
    try {
      this.bgm.stop();
      this.bgm.destroy();
    } catch (e) {
      /* ignore */
    }
    this.bgm = null;
  }
  /** 雨声 / 鸟鸣。音量压在敲击音和佛乐下面,可各自开关 */
  setAmbience(rain: boolean, bird: boolean): void {
    this.rainOn = rain;
    this.birdOn = bird;
    this.syncAmbience();
  }
  pauseAmbience(): void {
    this.ambiencePaused = true;
    this.stopAmbienceLoops();
  }
  resumeAmbience(): void {
    this.ambiencePaused = false;
    this.syncAmbience();
  }
  private syncAmbience(): void {
    if (!this.soundOn || !this.ctx || !this.unlocked || this.ambiencePaused) {
      this.stopAmbienceLoops();
      return;
    }
    if (this.rainOn) this.ensureLoop('rain');
    else this.stopLoop('rain');
    if (this.birdOn) this.ensureLoop('bird');
    else this.stopLoop('bird');
  }
  private ensureLoop(kind: 'rain' | 'bird'): void {
    if (kind === 'rain' ? this.rainSrc : this.birdSrc) return;
    try {
      const sr = this.ctx.sampleRate || 44100;
      const len = sr * 2;
```

```
      const buffer = this.ctx.createBuffer(1, len, sr);
      const data = buffer.getChannelData(0);
      if (kind === 'rain') this.fillRain(data);
      else this.fillBird(data, sr);
      const src = this.ctx.createBufferSource();
      src.buffer = buffer;
      src.loop = true;
      const gain = this.ctx.createGain();
      gain.gain.value = kind === 'rain' ? 0.045 : 0.03;
      src.connect(gain);
      gain.connect(this.ctx.destination);
      src.start();
      if (kind === 'rain') this.rainSrc = src;
      else this.birdSrc = src;
    } catch (e) {
      /* 白噪音失败不影响敲击 */
    }
  }
  private fillRain(data: Float32Array): void {
    let prev = 0;
    for (let i = 0; i < data.length; i++) {
      const white = Math.random() * 2 - 1;
      prev = prev * 0.85 + white * 0.15;
      data[i] = prev;
    }
  }
  private fillBird(data: Float32Array, sr: number): void {
    const chirps = [0.15, 0.7, 1.25, 1.7];
    for (const start of chirps) {
      const from = Math.floor(start * sr);
      const n = Math.floor(0.08 * sr);
      for (let i = 0; i < n && from + i < data.length; i++) {
        const t = i / sr;
        const env = Math.sin((i / n) * Math.PI);
        const f = 2800 - t * 900;
        data[from + i] += Math.sin(2 * Math.PI * f * t) * env * 0.8;
      }
    }
  }
  private stopAmbienceLoops(): void {
    this.stopLoop('rain');
    this.stopLoop('bird');
  }
  private stopLoop(kind: 'rain' | 'bird'): void {
    const src = kind === 'rain' ? this.rainSrc : this.birdSrc;
    if (!src) return;
    try { src.stop(); } catch (e) { /* already stopped */ }
    if (kind === 'rain') this.rainSrc = null;
    else this.birdSrc = null;
  }
```

```
  pauseBgm(): void {
    if (this.bgm) {
      try {
        this.bgm.pause();
      } catch (e) {
        /* ignore */
      }
    }
  }
  resumeBgm(): void {
    if (this.bgm && this.soundOn) {
      try {
        this.bgm.play();
      } catch (e) {
        /* ignore */
      }
    }
  }
  private fadeTo(target: number, ms: number): void {
    if (!this.bgm) return;
    if (this.fadeTimer) clearInterval(this.fadeTimer);
    const steps = Math.max(1, Math.floor(ms / 50));
    let i = 0;
    this.fadeTimer = setInterval(() => {
      i++;
      if (!this.bgm) {
        clearInterval(this.fadeTimer);
        this.fadeTimer = null;
        return;
      }
      this.bgm.volume = Math.min(1, (target * i) / steps);
      if (i >= steps) {
        clearInterval(this.fadeTimer);
        this.fadeTimer = null;
      }
    }, 50);
  }
  get currentBgmId(): string {
    return this.bgmId;
  }
}
// -------- src/ui/Node.ts --------
// 自研轻量 UI 树:节点基类(绝对定位、命中测试、触摸分发)
export type TouchPhase = 'start' | 'move' | 'end';
export abstract class UINode {
  x = 0;
  y = 0;
  w = 0;
  h = 0;
  visible = true;
```

```
  children: UINode[] = [];
  parent: UINode | null = null;
  add<T extends UINode>(child: T): T {
    child.parent = this;
    this.children.push(child);
    return child;
  }
  remove(child: UINode): void {
    const i = this.children.indexOf(child);
    if (i >= 0) this.children.splice(i, 1);
  }
  removeAll(): void {
    this.children.length = 0;
  }
  get absX(): number {
    return (this.parent ? this.parent.absX : 0) + this.x;
  }
  get absY(): number {
    return (this.parent ? this.parent.absY : 0) + this.y;
  }
  hitTest(px: number, py: number): boolean {
    return (
      this.visible &&
      px >= this.absX &&
      px <= this.absX + this.w &&
      py >= this.absY &&
      py <= this.absY + this.h
    );
  }
  /** 子类实现绘制 */
  protected abstract draw(ctx: CanvasRenderingContext2D): void;
  render(ctx: CanvasRenderingContext2D): void {
    if (!this.visible) return;
    ctx.save();
    this.draw(ctx);
    for (const c of this.children) c.render(ctx);
    ctx.restore();
  }
  /** 触摸分发:后添加的子节点优先(顶层优先) */
  dispatchTouch(phase: TouchPhase, px: number, py: number): boolean {
    if (!this.visible) return false;
    for (let i = this.children.length - 1; i >= 0; i--) {
      if (this.children[i].dispatchTouch(phase, px, py)) return true;
    }
    return this.onTouch(phase, px, py);
  }
  protected onTouch(_phase: TouchPhase, _px: number, _py: number): boolean {
    return false;
  }
}
```

```
/** 容器节点(自身不绘制) */
export class UIContainer extends UINode {
  protected draw(): void {}
}
// -------- src/ui/widgets.ts --------
// 基础控件:Label / Button / Panel / Toggle
import { UINode, TouchPhase } from './Node';
export function roundRect(
  ctx: CanvasRenderingContext2D,
  x: number,
  y: number,
  w: number,
  h: number,
  r: number
): void {
  const rr = Math.min(r, w / 2, h / 2);
  ctx.beginPath();
  ctx.moveTo(x + rr, y);
  ctx.arcTo(x + w, y, x + w, y + h, rr);
  ctx.arcTo(x + w, y + h, x, y + h, rr);
  ctx.arcTo(x, y + h, x, y, rr);
  ctx.arcTo(x, y, x + w, y, rr);
  ctx.closePath();
}
export class Label extends UINode {
  text = '';
  font = 16;
  color = '#FFFFFF';
  align: CanvasTextAlign = 'center';
  bold = false;
  alpha = 1;
  constructor(text = '', font = 16, color = '#FFFFFF') {
    super();
    this.text = text;
    this.font = font;
    this.color = color;
  }
  protected draw(ctx: CanvasRenderingContext2D): void {
    ctx.globalAlpha = this.alpha;
    ctx.fillStyle = this.color;
    ctx.font = `${this.bold ? 'bold ' : ''}${this.font}px sans-serif`;
    ctx.textAlign = this.align;
    ctx.textBaseline = 'middle';
    ctx.fillText(this.text, this.absX, this.absY);
    ctx.globalAlpha = 1;
  }
}
export interface ButtonStyle {
  bg: string;
  bgPressed: string;
```

```
  textColor: string;
  font?: number;
  radius?: number;
  border?: string;
}
export const defaultBtnStyle: ButtonStyle = {
  bg: 'rgba(232,184,75,0.16)',
  bgPressed: 'rgba(232,184,75,0.35)',
  textColor: '#E8B84B',
  font: 15,
  radius: 20,
  border: 'rgba(232,184,75,0.5)',
};
export class Button extends UINode {
  label: string;
  style: ButtonStyle;
  onTap: (() => void) | null = null;
  enabled = true;
  private pressed = false;
  constructor(label: string, w: number, h: number, style?: Partial<ButtonStyle>) {
    super();
    this.label = label;
    this.w = w;
    this.h = h;
    this.style = { ...defaultBtnStyle, ...(style || {}) };
  }
  protected draw(ctx: CanvasRenderingContext2D): void {
    const { absX, absY, w, h } = this;
    const s = this.style;
    ctx.globalAlpha = this.enabled ? 1 : 0.4;
    ctx.fillStyle = this.pressed ? s.bgPressed : s.bg;
    roundRect(ctx, absX, absY, w, h, s.radius ?? 20);
    ctx.fill();
    if (s.border) {
      ctx.strokeStyle = s.border;
      ctx.lineWidth = 1;
      ctx.stroke();
    }
    ctx.fillStyle = s.textColor;
    ctx.font = `${s.font ?? 15}px sans-serif`;
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText(this.label, absX + w / 2, absY + h / 2 + 1);
    ctx.globalAlpha = 1;
  }
  protected onTouch(phase: TouchPhase, px: number, py: number): boolean {
    if (!this.enabled) return this.hitTest(px, py);
    if (phase === 'start' && this.hitTest(px, py)) {
      this.pressed = true;
      return true;
```

```
    }
    if (phase === 'end' && this.pressed) {
      this.pressed = false;
      if (this.hitTest(px, py)) this.onTap?.();
      return true;
    }
    if (phase === 'move' && this.pressed) return true;
    return false;
  }
}
export class Panel extends UINode {
  bg = 'rgba(13,20,32,0.92)';
  border = 'rgba(232,184,75,0.35)';
  radius = 16;
  protected draw(ctx: CanvasRenderingContext2D): void {
    ctx.fillStyle = this.bg;
    roundRect(ctx, this.absX, this.absY, this.w, this.h, this.radius);
    ctx.fill();
    ctx.strokeStyle = this.border;
    ctx.lineWidth = 1;
    ctx.stroke();
  }
}
/** 开关控件 */
export class Toggle extends UINode {
  value = true;
  label = '';
  onChange: ((v: boolean) => void) | null = null;
  private pressed = false;
  constructor(label: string, value: boolean, w = 200, h = 44) {
    super();
    this.label = label;
    this.value = value;
    this.w = w;
    this.h = h;
  }
  protected draw(ctx: CanvasRenderingContext2D): void {
    const { absX, absY, h, w } = this;
    ctx.fillStyle = '#F5EDD8';
    ctx.font = '15px sans-serif';
    ctx.textAlign = 'left';
    ctx.textBaseline = 'middle';
    ctx.fillText(this.label, absX, absY + h / 2);
    // 开关
    const sw = 46;
    const sh = 26;
    const sx = absX + w - sw;
    const sy = absY + (h - sh) / 2;
    ctx.fillStyle = this.value ? 'rgba(232,184,75,0.8)' : 'rgba(255,255,255,0.15)';
    roundRect(ctx, sx, sy, sw, sh, sh / 2);
```

```
    ctx.fill();
    ctx.fillStyle = '#FFF';
    ctx.beginPath();
    ctx.arc(sx + (this.value ? sw - sh / 2 : sh / 2), sy + sh / 2, sh / 2 - 3, 0, Math.PI * 2);
    ctx.fill();
  }
  protected onTouch(phase: TouchPhase, px: number, py: number): boolean {
    if (phase === 'start' && this.hitTest(px, py)) {
      this.pressed = true;
      return true;
    }
    if (phase === 'end' && this.pressed) {
      this.pressed = false;
      if (this.hitTest(px, py)) {
        this.value = !this.value;
        this.onChange?.(this.value);
      }
      return true;
    }
    return false;
  }
}
// -------- src/ui/Toast.ts --------
// 全局 Toast(轻提示),由 Game 渲染在场景之上
import { roundRect } from './widgets';
interface ToastItem {
  text: string;
  until: number;
}
export class Toast {
  private items: ToastItem[] = [];
  show(text: string, durationMs = 1800): void {
    this.items.push({ text, until: Date.now() + durationMs });
    if (this.items.length > 3) this.items.shift();
  }
  render(ctx: CanvasRenderingContext2D, screenW: number, screenH: number): void {
    const now = Date.now();
    this.items = this.items.filter((t) => t.until > now);
    if (!this.items.length) return;
    ctx.save();
    ctx.font = '14px sans-serif';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    let y = screenH * 0.68;
    for (const item of this.items) {
      const remain = item.until - now;
      const alpha = Math.min(1, remain / 300);
      const metrics = ctx.measureText(item.text);
      const w = metrics.width + 36;
      ctx.globalAlpha = alpha * 0.85;
```

```
      ctx.fillStyle = 'rgba(0,0,0,0.75)';
      roundRect(ctx, (screenW - w) / 2, y - 16, w, 32, 16);
      ctx.fill();
      ctx.globalAlpha = alpha;
      ctx.fillStyle = '#F5EDD8';
      ctx.fillText(item.text, screenW / 2, y);
      y -= 40;
    }
    ctx.restore();
  }
}
// -------- src/scenes/BootScene.ts --------
// 启动场景:资源自检 + 云登录等待 + 过渡动画
// 设计:不阻塞!云登录最多等 2.5s,超时直接进主页(游客模式)
import { Scene } from '../core/Scene';
import { STRINGS } from '../data/strings';
import { HomeScene } from './HomeScene';
import type { Game } from '../Game';
export class BootScene extends Scene {
  private progress = 0;
  private dots = 0;
  private startTime = 0;
  private entered = false;
  constructor(game: Game) {
    super(game);
  }
  enter(): void {
    this.startTime = Date.now();
    // 动画进度条(假进度,真等待在 sync.init 里)
    this.game.ticker.to(2000, (t) => {
      this.progress = t;
    });
    // 最长 2.5s 必进主页(弱网首启容错)
    this.game.ticker.delay(2500, () => this.goHome());
  }
  private goHome(): void {
    if (this.entered) return;
    this.entered = true;
    this.game.scenes.replace(new HomeScene(this.game));
  }
  exit(): void {}
  update(dt: number): void {
    this.dots = Math.floor((Date.now() - this.startTime) / 400) % 4;
    // 提前登录成功且进度走完 → 提前进
    if (this.progress >= 1 && this.game.sync.state === 'online') this.goHome();
  }
  render(ctx: CanvasRenderingContext2D): void {
    const { width, height } = this.game.renderer;
    // 背景
    const grad = ctx.createLinearGradient(0, 0, 0, height);
```

```
    grad.addColorStop(0, '#1A2332');
    grad.addColorStop(1, '#0D1420');
    ctx.fillStyle = grad;
    ctx.fillRect(0, 0, width, height);
    const cx = width / 2;
    const cy = height / 2;
    // 木鱼剪影
    ctx.fillStyle = 'rgba(232,184,75,0.12)';
    ctx.beginPath();
    ctx.arc(cx, cy - 60, 80, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = '#E8B84B';
    ctx.font = 'bold 56px sans-serif';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText(STRINGS.title, cx, cy - 50);
    ctx.fillStyle = '#9A8F74';
    ctx.font = '16px sans-serif';
    ctx.fillText(STRINGS.subtitle, cx, cy + 20);
    // 进度条
    const barW = width * 0.5;
    const barH = 4;
    const barX = cx - barW / 2;
    const barY = cy + 80;
    ctx.fillStyle = 'rgba(232,184,75,0.15)';
    ctx.fillRect(barX, barY, barW, barH);
    ctx.fillStyle = '#E8B84B';
    ctx.fillRect(barX, barY, barW * this.progress, barH);
    // 加载文案
    ctx.fillStyle = '#9A8F74';
    ctx.font = '13px sans-serif';
    ctx.fillText('正在静心' + '.'.repeat(this.dots), cx, barY + 30);
  }
}
// -------- src/scenes/HomeScene.ts --------
// 主场景:木鱼敲击(本游戏 90% 的体验在这里)
// 职责:敲击判定 → 三路反馈(视/听/触)→ 飘字/涟漪 → 连击 → 突破演出 → 弹层
import { Scene } from '../core/Scene';
import { bus, Events } from '../core/EventBus';
import { FEEL, OFFLINE_UNLOCK_LEVEL } from '../data/configs';
import { STRINGS, COMBO_TEXTS, TUTORIAL_STEPS } from '../data/strings';
import { fmtNumber, fmtDuration, clamp, todayKey } from '../utils/format';
import { COPY_HIT_R, PRACTICE_REWARD, type LessonDef } from '../data/practice';
import {
  applyCopy,
  applyOnceTap,
  applySettle,
  copyCharOf,
  copyDotOffset,
  dayIndexOf,
```

```
  lessonOf,
  listenReady,
  notchCount,
  onceReady,
  practiceLabel,
  ringCount,
  ringRadiusRatio,
  verseOf,
} from '../utils/practiceRules';
import { ObjectPool } from '../utils/pool';
import { UIContainer } from '../ui/Node';
import { Button, Panel, Label, roundRect } from '../ui/widgets';
import { CultivateScene } from './CultivateScene';
import { DailyScene } from './DailyScene';
import { RankScene } from './RankScene';
import { SettingScene } from './SettingScene';
import { SutraScene } from './SutraScene';
import { ENERGY_MAX } from '../data/play';
import type { Game } from '../Game';
interface FloatText {
  x: number; y: number; vy: number;
  text: string; color: string; size: number;
  life: number; maxLife: number; active: boolean;
}
interface Ripple {
  x: number; y: number; r: number; maxR: number;
  life: number; maxLife: number; active: boolean;
}
export class HomeScene extends Scene {
  private ui = new UIContainer();
  private unsubscribe: (() => void)[] = [];
  // 木鱼状态
  private fishY = 0;
  private fishR = 0;
  private squash = 1;          // scaleY 按压回弹
  private glowPulse = 0;       // 常驻呼吸光晕
  // 特效池
  private floats: FloatText[] = [];
  private floatPool = new ObjectPool<FloatText>(
    () => ({ x: 0, y: 0, vy: 0, text: '', color: '', size: 16, life: 0, maxLife: 0, active: false }),
    (o) => { o.active = false; }
  );
  private ripples: Ripple[] = [];
  private ripplePool = new ObjectPool<Ripple>(
    () => ({ x: 0, y: 0, r: 0, maxR: 0, life: 0, maxLife: 0, active: false }),
    (o) => { o.active = false; }
  );
  // 弹层
  private showOffline = false;
  private offlineInfo: { merit: number; hours: number } | null = null;
```

```
  private offlineDoubleBtn: { x: number; y: number; w: number; h: number } | null = null;
  private showBreakthrough = false;
  private breakthroughInfo: { name: string; unlocks: string[] } | null = null;
  private showTutorial = false;
  private tutorialStep = 0;
  private dock = { x: 0, y: 0, w: 0, h: 0 };
  /** 今日修行进行中。只放内存，冷启动和「先不修」都回到普通敲击 */
  private practicing = false;
  private lastSettleAt = 0;
  private onceWaitMs = 0;
  private copyShake = 0;
  private verse = '';
  private practiceRect = { x: 0, y: 0, w: 0, h: 0 };
  private pauseRect = { x: 0, y: 0, w: 0, h: 0 };
  // 顶部 UI 引用
  private meritLabel!: Label;
  private levelLabel!: Label;
  private syncTipLabel!: Label;
  private meritDoubleBtn!: Button;
  constructor(game: Game) {
    super(game);
  }
  enter(): void {
    const { width, height, contentBottom } = this.game.renderer;
    this.fishY = height * 0.52;
    this.fishR = Math.min(width * 0.22, 90);
    this.buildUI();
    this.bindEvents();
    // 新手引导
    if (!this.game.save.guided) {
      this.showTutorial = true;
      this.tutorialStep = 0;
    }
    // 离线收益检查(罗汉解锁 + 离线 ≥3 分钟)
    this.checkOfflineReward();
    // BGM 恢复
    if (this.game.save.bgmId !== 'none') {
      this.game.audio.playBgm(this.game.save.bgmId);
    }
  }
  exit(): void {
    this.unsubscribe.forEach((fn) => fn());
    this.unsubscribe = [];
    this.ui.removeAll();
  }
  // ---------- UI 搭建 ----------
  private buildUI(): void {
    const { width, contentTop, contentBottom } = this.game.renderer;
    const scene = this.game.skin.currentScene;
    // 顶部:功德数
```

```
    this.meritLabel = new Label('0', 34, scene.textMain);
    this.meritLabel.bold = true;
    this.meritLabel.x = width / 2;
    this.meritLabel.y = contentTop + 30;
    this.ui.add(this.meritLabel);
    // 境界 + 进度
    this.levelLabel = new Label('', 14, scene.textSub);
    this.levelLabel.x = width / 2;
    this.levelLabel.y = contentTop + 62;
    this.ui.add(this.levelLabel);
    // 同步状态提示
    this.syncTipLabel = new Label('', 11, '#7A6F55');
    this.syncTipLabel.x = width / 2;
    this.syncTipLabel.y = contentTop + 84;
    this.ui.add(this.syncTipLabel);
      this.buildDock(width, contentBottom);
    // 分享按钮(右上)
    const shareBtn = new Button(STRINGS.shareBtn, 96, 34, { font: 13 });
    shareBtn.x = width - 106;
    shareBtn.y = contentTop + 8;
    shareBtn.onTap = () => this.game.share.share('button');
    this.ui.add(shareBtn);
    // 静心加成按钮(左上,激励视频:10 分钟内功德翻倍)
    this.meritDoubleBtn = new Button(STRINGS.adMeritDouble, 96, 34, { font: 13 });
    this.meritDoubleBtn.x = 10;
    this.meritDoubleBtn.y = contentTop + 8;
    this.meritDoubleBtn.onTap = () => {
      this.game.ad.show('merit_double', () => {
        this.game.save.extra.meritDoubleUntil = Date.now() + 10 * 60 * 1000;
        this.game.saveManager.markDirty();
        this.game.toast.show('静心加成:10 分钟内功德翻倍');
        this.refreshHUD();
      });
    };
    this.ui.add(this.meritDoubleBtn);
    this.refreshHUD();
  }
  /** 底部六键收成一块底栏,两行三列对齐 */
  private buildDock(width: number, contentBottom: number): void {
    const cols = 3;
    const gap = 8;
    const inset = 12;
    const btnH = 36;
    const margin = 14;
    const innerW = width - margin * 2 - inset * 2;
    const btnW = (innerW - gap * (cols - 1)) / cols;
    const dockH = inset * 2 + btnH * 2 + gap;
    const dockY = contentBottom - dockH;
    this.dock = { x: margin, y: dockY, w: width - margin * 2, h: dockH };
    const quiet = {
```

```
      bg: 'rgba(255,255,255,0.05)',
      bgPressed: 'rgba(232,184,75,0.28)',
      textColor: '#E4D7B8',
      border: 'rgba(232,184,75,0.22)',
      font: 14,
      radius: 12,
    };
    const accent = {
      ...quiet,
      bg: 'rgba(232,184,75,0.2)',
      textColor: '#F0D48A',
      border: 'rgba(232,184,75,0.7)',
    };
    const items: Array<{ label: string; accent?: boolean; onTap: () => void }> = [
      { label: '听经', accent: true, onTap: () => this.game.scenes.push(new SutraScene(this.game)) },
      {
        label: '真言',
        onTap: () => {
          const text = this.game.gongfa.cast();
          this.game.toast.show(text || `能量 ${this.game.gongfa.play.energy}/${ENERGY_MAX}`);
        },
      },
      { label: STRINGS.navCultivate, onTap: () => this.game.scenes.push(new CultivateScene(this.game)) },
      { label: STRINGS.navDaily, onTap: () => this.game.scenes.push(new DailyScene(this.game)) },
      { label: STRINGS.navRank, onTap: () => this.game.scenes.push(new RankScene(this.game)) },
      { label: STRINGS.navSettings, onTap: () => this.game.scenes.push(new SettingScene(this.game)) },
    ];
    items.forEach((item, i) => {
      const col = i % cols;
      const row = Math.floor(i / cols);
      const btn = new Button(item.label, btnW, btnH, item.accent ? accent : quiet);
      btn.x = margin + inset + col * (btnW + gap);
      btn.y = dockY + inset + row * (btnH + gap);
      btn.onTap = item.onTap;
      this.ui.add(btn);
    });
  }
  private drawDock(ctx: CanvasRenderingContext2D): void {
    const { x, y, w, h } = this.dock;
    ctx.fillStyle = 'rgba(16, 22, 30, 0.88)';
    roundRect(ctx, x, y, w, h, 18);
    ctx.fill();
    ctx.strokeStyle = 'rgba(232,184,75,0.28)';
    ctx.lineWidth = 1;
    ctx.stroke();
  }
  private refreshHUD(): void {
    const save = this.game.save;
    const level = this.game.levelSystem.current;
    const next = this.game.levelSystem.next;
```

```
      ctx.font = 'bold 20px sans-serif';
      ctx.fillText(STRINGS.title, W / 2, H - 36);
      return canvas.toTempFilePathSync({
        x: 0, y: 0, width: W, height: H,
        destWidth: W, destHeight: H,
      });
    } catch (e) {
      console.warn('[Share] 分享卡绘制失败:', e);
      return '';
    }
  }
  private handleShared(): void {
    this.game.daily.claimShareMerit().then((ok) => {
      if (ok) this.game.toast.show('分享成功,功德 +100');
    });
  }
}
// -------- src/systems/AdSystem.ts --------
// 激励视频广告:封装 + 频控 + 降级
// 所有广告入口均为主动领取按钮,绝不打断敲击
import { AD_UNIT_ID, AD_LIMITS, AD_DAILY_TOTAL, AD_GRACE_MS, DEV_MOCK_ADS } from '../data/configs';
import { STRINGS } from '../data/strings';
import type { Game } from '../Game';
export type AdTag = 'merit_double' | 'offline_double' | 'skin_trial';
export class AdSystem {
  private game: Game;
  private ad: any = null;
  private loading = false;
  constructor(game: Game) {
    this.game = game;
    this.initAd();
  }
  private initAd(): void {
    if (DEV_MOCK_ADS || !AD_UNIT_ID) return; // 开发期 mock
    try {
      this.ad = wx.createRewardedVideoAd({ adUnitId: AD_UNIT_ID });
      this.ad.onError((e: any) => console.warn('[Ad] 错误:', e));
    } catch (e) {
      console.warn('[Ad] 初始化失败:', e);
    }
  }
  /** 当日剩余次数(含总频控) */
  remain(tag: AdTag): number {
    if (this.game.daily.adTotalToday() >= AD_DAILY_TOTAL) return 0;
    return this.game.daily.adRemain(tag, AD_LIMITS[tag] || 1);
  }
  /**
   * 展示激励视频
   * onReward 在完整看完后回调;取消/失败不扣次数
   */
```

```
  show(tag: AdTag, onReward: () => void): void {
    if (Date.now() - this.game.bootAt < AD_GRACE_MS) {
      this.game.toast.show(STRINGS.adGrace);
      return;
    }
    if (this.remain(tag) <= 0) {
      this.game.toast.show(STRINGS.adLimitReached);
      return;
    }
    const grant = () => {
      this.game.daily.recordAd(tag).then((ok) => {
        if (!ok) {
          this.game.toast.show(STRINGS.adLimitReached);
          return;
        }
        onReward();
      });
    };
    // 开发期 mock:直接发奖励(提审前必须接真实广告)
    if (DEV_MOCK_ADS || !this.ad) {
      grant();
      return;
    }
    if (this.loading) {
      this.game.toast.show(STRINGS.adNotReady);
      return;
    }
    this.loading = true;
    const onClose = (res: any) => {
      this.ad.offClose(onClose);
      this.loading = false;
      if (res && res.isEnded) {
        grant();
      } else {
        this.game.toast.show('未看完,未获得奖励');
      }
    };
    this.ad.onClose(onClose);
    this.ad.show().catch(() => {
      this.ad.offClose(onClose);
      this.loading = false;
      this.ad
        .load()
        .then(() => {
          this.loading = true;
          this.ad.onClose(onClose);
          return this.ad.show();
        })
        .catch((e: any) => {
          this.loading = false;
```

```
          console.warn('[Ad] 展示失败:', e);
          this.game.toast.show(STRINGS.adNotReady);
        });
    });
  }
}
// -------- src/systems/Analytics.ts --------
// 埋点:批量写云数据库 events 集合(30s 一批,失败丢弃不阻塞游戏)
import type { Game } from '../Game';
interface TrackEvent {
  event: string;
  ts: number;
  duration?: number;
  extra?: Record<string, any>;
}
export class Analytics {
  private game: Game;
  private queue: TrackEvent[] = [];
  private timer: any = null;
  private sessionStart = Date.now();
  constructor(game: Game) {
    this.game = game;
  }
  /** 启动定时上报(可在 onShow 重建) */
  init(): void {
    if (this.timer) return;
    this.sessionStart = Date.now();
    this.timer = setInterval(() => this.flush(), 30000);
  }
  track(event: string, extra?: Record<string, any>, duration?: number): void {
    this.queue.push({ event, ts: Date.now(), duration, extra });
    if (this.queue.length >= 20) this.flush();
  }
  /** 关键事件:启动/敲击时长/突破/功课/广告/分享 */
  sessionEnd(): void {
    this.track('session', undefined, Math.floor((Date.now() - this.sessionStart) / 1000));
    this.flush();
  }
  private async flush(): Promise<void> {
    if (!this.queue.length) return;
    if (this.game.sync.state !== 'online') {
      this.queue.length = 0; // 游客模式丢弃,避免积压
      return;
    }
    const batch = this.queue.splice(0, this.queue.length);
    try {
      const db = wx.cloud.database();
      // 逐条写入(events 集合权限:仅创建者可读写)
      await Promise.all(
        batch.map((e) =>
```

```
          db.collection('events').add({
            data: { ...e, _openid: undefined }, // _openid 由云端注入
          })
        )
      );
    } catch (e) {
      console.warn('[Analytics] flush 失败:', e);
    }
  }
  destroy(): void {
    if (this.timer) {
      clearInterval(this.timer);
      this.timer = null;
    }
  }
}
// -------- src/systems/playRules.ts --------
import type { SaveData } from '../data/types';
import { CHARTS, NOTE_CAP, type ChartDef, type GradeId, type Note } from '../data/play';
export interface PlayState {
  learned: string[];
  slots: string[];
  mastered: string[];
  mantra: string;
  energy: number;
  bodhi: number;
  relics: number;
  cycle: number;
  levelPeak: number;
  grades: Record<string, string>;
  pendingRebirth: boolean;
}
const GRADE_RANK: Record<string, number> = { fail: 1, entry: 2, insight: 3, supreme: 4 };
export function ensurePlay(save: SaveData): PlayState {
  const raw = (save.extra.play || {}) as Partial<PlayState>;
  const play: PlayState = {
    learned: Array.isArray(raw.learned) ? raw.learned : [],
    slots: Array.isArray(raw.slots) ? raw.slots : [],
    mastered: Array.isArray(raw.mastered) ? raw.mastered : [],
    mantra: raw.mantra || '',
    energy: raw.energy || 0,
    bodhi: raw.bodhi || 0,
    relics: raw.relics || 0,
    cycle: raw.cycle || 0,
    levelPeak: raw.levelPeak || 0,
    grades: raw.grades || {},
    pendingRebirth: !!raw.pendingRebirth,
  };
  save.extra.play = play;
  return play;
```

```
}
export function buildChart(def: ChartDef): Note[] {
  const notes: Note[] = [];
  let t = 1;
  let i = 0;
  while (t < def.duration - 0.25 && notes.length < NOTE_CAP) {
    const type = def.cycle[i % def.cycle.length];
    const pos = 0.22 + ((i * 47) % 56) / 100;
    const dur = type === 'tap' ? undefined : Math.max(0.28, def.gap * 0.85);
    const to = type === 'slide'
      ? (pos > 0.5 ? Math.max(0.18, pos - 0.28) : Math.min(0.82, pos + 0.28))
      : undefined;
    notes.push({ t: round3(t), type, pos: round3(pos), dur, to });
    t += def.gap + (type === 'tap' ? 0 : (dur || 0) * 0.45);
    i += 1;
  }
  return notes;
}
export function judgeDelta(deltaMs: number, perfectMs: number, goodMs: number): 'perfect' | 'good' | 'miss' {
  const d = Math.abs(deltaMs);
  if (d <= perfectMs) return 'perfect';
  if (d <= goodMs) return 'good';
  return 'miss';
}
export function gradeOf(perfect: number, good: number, miss: number): GradeId {
  const total = perfect + good + miss;
  if (total <= 0) return 'fail';
  const hit = (perfect + good) / total;
  if (miss === 0 && perfect / total >= 0.95) return 'supreme';
  if (hit >= 0.95) return 'insight';
  if (hit >= 0.8) return 'entry';
  return 'fail';
}
export function betterGrade(a: string, b: string): string {
  return (GRADE_RANK[a] || 0) >= (GRADE_RANK[b] || 0) ? a : b;
}
/** 另一台设备的功法进度并进来,不覆盖更高的评价 */
export function mergePlay(local: PlayState, cloud: Partial<PlayState> | null | undefined): void {
  if (!cloud) return;
  const learned = new Set([...(local.learned || []), ...((cloud.learned as string[]) || [])]);
  local.learned = Array.from(learned);
  const mastered = new Set([...(local.mastered || []), ...((cloud.mastered as string[]) || [])]);
  local.mastered = Array.from(mastered).filter((id) => local.learned.includes(id));
  const grades = { ...(cloud.grades || {}) };
  Object.keys(local.grades || {}).forEach((id) => {
    grades[id] = betterGrade(local.grades[id], grades[id] || '');
  });
  local.grades = grades;
  if (!local.slots.length && Array.isArray(cloud.slots)) local.slots = cloud.slots.filter((id) => local.learne
d.includes(id));
```

```
  if (!local.mantra && cloud.mantra) local.mantra = cloud.mantra;
  local.bodhi = Math.max(local.bodhi || 0, cloud.bodhi || 0);
  local.energy = Math.max(local.energy || 0, cloud.energy || 0);
  local.levelPeak = Math.max(local.levelPeak || 0, cloud.levelPeak || 0);
}
function round3(n: number): number {
  return Math.round(n * 1000) / 1000;
}
export function selfCheckPlayRules(): void {
  const chart = CHARTS[0];
  const notes = buildChart(chart);
  if (notes.length < 8 || notes.length > NOTE_CAP) throw new Error('chart length');
  for (let i = 1; i < notes.length; i++) {
    if (notes[i].t - notes[i - 1].t < chart.gap - 0.001) throw new Error('notes too dense');
  }
  if (judgeDelta(40, 80, 140) !== 'perfect') throw new Error('perfect');
  if (judgeDelta(100, 80, 140) !== 'good') throw new Error('good');
  if (judgeDelta(200, 80, 140) !== 'miss') throw new Error('miss');
  if (gradeOf(10, 0, 0) !== 'supreme') throw new Error('supreme');
  if (gradeOf(9, 1, 0) !== 'insight') throw new Error('insight');
  if (gradeOf(8, 0, 2) !== 'entry') throw new Error('entry');
  if (gradeOf(5, 0, 5) !== 'fail') throw new Error('fail');
  if (betterGrade('entry', 'supreme') !== 'supreme') throw new Error('rank');
}
// -------- src/data/types.ts --------
// 存档数据模型(本地镜像与云端 profiles 保持一致)
export interface DailyState {
  dateKey: string;          // YYYY-MM-DD(UTC+8)
  taps: number;             // 今日敲击数
  claimed: boolean;         // 今日功课宝箱是否已领
  streak: number;           // 连续完成天数
  shareMeritClaimed: boolean; // 今日分享功德是否已领
  adWatch: Record<string, number>; // 广告点位当日观看次数
  practiceProgress: number; // 今日修行进度。静听为已持续毫秒，其余为已完成下数
  practiceDone: boolean;
  practiceStreak: number;  // 连续圆满天数，与功课 streak 分开
}
export interface Inventory {
  skins: string[];
  scenes: string[];
  bgms: string[];
}
export interface SaveData {
  version: number;
  merit: number;            // 累计功德
  totalTaps: number;        // 累计敲击
  skinId: string;
  sceneId: string;
  bgmId: string;
  soundOn: boolean;
```

```
  vibrateOn: boolean;
  tapSound: string;         // 木鱼音色: 'resonant'(腔体共振) | 'wooden'(木质敲击) | 'crisp'(清脆实木) | 'thump'(咚咚木鱼)
  inventory: Inventory;
  daily: DailyState;
  lastSeenAt: number;       // 上次离开时间戳(ms)
  lastSyncAt: number;       // 上次云同步时间戳(ms)
  offlineClaimedAt: number; // 上次离线收益领取时间戳(ms)
  pendingTaps: number;      // 未上报云端的敲击数
  pendingMerit: number;     // 未上报云端的功德增量
  guided: boolean;          // 新手引导已完成
  nickname: string;
  extra: Record<string, any>; // 二期扩展位(功法/舍利子/事件)
}
export const SAVE_VERSION = 2;
export function createDefaultSave(): SaveData {
  const now = Date.now();
  return {
    version: SAVE_VERSION,
    merit: 0,
    totalTaps: 0,
    skinId: 'classic_wood',
    sceneId: 'temple',
    bgmId: 'none',
    soundOn: true,
    vibrateOn: true,
    tapSound: 'resonant',
    inventory: { skins: ['classic_wood'], scenes: ['temple'], bgms: [] },
    daily: {
      dateKey: '',
      taps: 0,
      claimed: false,
      streak: 0,
      shareMeritClaimed: false,
      adWatch: {},
      practiceProgress: 0,
      practiceDone: false,
      practiceStreak: 0,
    },
    lastSeenAt: now,
    lastSyncAt: 0,
    offlineClaimedAt: 0,
    pendingTaps: 0,
    pendingMerit: 0,
    guided: false,
    nickname: '',
    extra: {},
  };
}
/** 旧版本存档迁移入口(按 version 逐级升级) */
```

```
export function migrateSave(data: any): SaveData {
  const def = createDefaultSave();
  if (!data || typeof data !== 'object') return def;
  // 浅合并 + 深合并关键子对象,缺失字段补默认
  const merged: SaveData = {
    ...def,
    ...data,
    inventory: { ...def.inventory, ...(data.inventory || {}) },
    daily: { ...def.daily, ...(data.daily || {}) },
    extra: { ...(data.extra || {}) },
  };
  merged.version = SAVE_VERSION;
  return merged;
}
// -------- src/data/configs.ts --------
// 全部数值与资源配置表(手感调参集中在这里,支持开发期热改)
// ---------- 云开发 ----------
export const CLOUD_ENV = 'kxcf-prod'; // 云环境 ID(部署时在云开发控制台创建)
// ---------- 境界 ----------
export interface LevelConfig {
  index: number;
  name: string;
  merit: number;        // 到达该境界所需累计功德
  unlocks: string[];    // 解锁内容描述(展示用)
}
export const LEVELS: LevelConfig[] = [
  { index: 0, name: '凡夫', merit: 0, unlocks: [] },
  { index: 1, name: '居士', merit: 1000, unlocks: ['皮肤「檀木」', '佛乐《心经》'] },
  { index: 2, name: '比丘', merit: 10000, unlocks: ['皮肤「冰种翡翠」', '佛乐《大悲咒》'] },
  { index: 3, name: '罗汉', merit: 100000, unlocks: ['离线禅修', '场景「竹林幽潭」', '佛乐《金刚经》'] },
  { index: 4, name: '菩萨', merit: 1000000, unlocks: ['称号飘字'] },
  { index: 5, name: '佛', merit: 10000000, unlocks: ['终极特效'] },
];
// ---------- 木鱼皮肤 ----------
export interface SkinConfig {
  id: string;
  name: string;
  bonus: number;          // 功德加成 0.05 = +5%
  unlockLevel: number;    // 解锁所需境界 index
  // 音色(真实木鱼合成参数:基频 420~650Hz,木质腔体共振)
  freq: number;           // 基频 Hz
  decay: number;          // 衰减秒
  wave: OscillatorType;
  // 配色
  body: string;           // 木鱼主体色
  bodyDark: string;       // 暗部
  highlight: string;      // 高光
  mouth: string;          // 鱼口
}
export const SKINS: SkinConfig[] = [
```

```
  {
    id: 'classic_wood', name: '经典木色', bonus: 0, unlockLevel: 0,
    freq: 480, decay: 0.32, wave: 'sine',
    body: '#8B5A2B', bodyDark: '#5D3A1A', highlight: '#C89B6D', mouth: '#3A2410',
  },
  {
    id: 'sandalwood', name: '檀木', bonus: 0.05, unlockLevel: 1,
    freq: 400, decay: 0.38, wave: 'sine',
    body: '#6B3A2A', bodyDark: '#452216', highlight: '#A67B5B', mouth: '#2E160C',
  },
  {
    id: 'jade', name: '冰种翡翠', bonus: 0.1, unlockLevel: 2,
    freq: 620, decay: 0.26, wave: 'sine',
    body: '#5FA98A', bodyDark: '#3A7A60', highlight: '#A8D8C0', mouth: '#24503C',
  },
  {
    id: 'mist', name: '薄雾', bonus: 0.12, unlockLevel: 99,
    freq: 540, decay: 0.3, wave: 'sine',
    body: '#7E93A8', bodyDark: '#4C6274', highlight: '#D5E2EC', mouth: '#2C3E4C',
  },
  {
    id: 'sunbird', name: '金乌', bonus: 0.15, unlockLevel: 99,
    freq: 360, decay: 0.42, wave: 'sine',
    body: '#C4782A', bodyDark: '#8A4E12', highlight: '#F0C27A', mouth: '#5C3208',
  },
];
// ---------- 场景 ----------
export interface SceneConfig {
  id: string;
  name: string;
  unlockLevel: number;
  bgTop: string;
  bgBottom: string;
  accent: string;          // 装饰色(灯焰/竹叶)
  textMain: string;        // 主文案色
  textSub: string;         // 次文案色
}
export const SCENES: SceneConfig[] = [
  {
    id: 'temple', name: '青灯古佛', unlockLevel: 0,
    bgTop: '#1A2332', bgBottom: '#0D1420', accent: '#E8B84B',
    textMain: '#F5EDD8', textSub: '#9A8F74',
  },
  {
    id: 'bamboo', name: '竹林幽潭', unlockLevel: 3,
    bgTop: '#16302A', bgBottom: '#0A1A16', accent: '#7BC496',
    textMain: '#E8F2E4', textSub: '#8AA893',
  },
  {
    id: 'ridge', name: '雪脊', unlockLevel: 4,
```

```
    bgTop: '#243044', bgBottom: '#121820', accent: '#D7E4EF',
    textMain: '#F4F7FA', textSub: '#9AABBA',
  },
];
// ---------- BGM(音频文件上传云存储后回填 url) ----------
export interface BgmConfig {
  id: string;
  name: string;
  unlockLevel: number;
  url: string; // 为空则不展示(素材未上传)
}
export const BGMS: BgmConfig[] = [
  { id: 'none', name: '无', unlockLevel: 0, url: '' },
  { id: 'xinjing', name: '心经', unlockLevel: 1, url: '' },
  { id: 'dabeizhou', name: '大悲咒', unlockLevel: 2, url: '' },
  { id: 'jingangjing', name: '金刚经', unlockLevel: 3, url: '' },
];
// ---------- 每日功课 ----------
export const DAILY_TAP_GOAL = 100;
export const DAILY_REWARDS = [66, 88, 108, 168]; // 宝箱随机功德
export const SHARE_MERIT = 100;                  // 每日首次分享双方功德
// ---------- 离线禅修 ----------
export const OFFLINE_TAPS_PER_HOUR = 300;
export const OFFLINE_CAP_HOURS = 12;
export const OFFLINE_LEVEL_COEFF = 0.2; // 每阶境界 +20%
export const OFFLINE_UNLOCK_LEVEL = 3;  // 罗汉解锁
// ---------- 云同步 ----------
export const SYNC_CONFIG = {
  loginTimeoutMs: 2500,   // 登录超时 → 游客模式
  tapBatchSize: 50,       // 攒够 N 次敲击触发上报
  flushIntervalMs: 30000, // 定时上报
  maxTapsPerSecond: 15,   // 服务端防刷频率上限
};
// ---------- 广告(流量主开通后回填 adUnitId;开发期 DEV_MOCK_ADS=true 直接发放奖励) ----------
export const DEV_MOCK_ADS = true;
export const AD_UNIT_ID = ''; // 例:adunit-xxxxxxxxxxxxxxx
export const AD_LIMITS: Record<string, number> = {
  merit_double: 3,    // 静心加成(今日功德翻倍)
  offline_double: 1,  // 离线收益翻倍
  skin_trial: 1,      // 皮肤试用
};
export const AD_DAILY_TOTAL = 6;
/** 首次进入后这段时间内不播激励视频 */
export const AD_GRACE_MS = 60_000;
/** 连续领满功课的天数，达成后给限定挂饰 */
export const STREAK_ORNAMENT_DAYS = 7;
export const STREAK_ORNAMENT_ID = 'jingjin';
// ---------- 排行榜 ----------
export const RANK_UPLOAD_INTERVAL_MS = 30000;
// ---------- 手感 ----------
```

```
export const FEEL = {
  tapMergeMs: 80,         // 该间隔内的重复 touchstart 合并(防抖动)
  squashScale: 0.9,       // 木鱼按压 scaleY
  squashMs: 100,          // 回弹时长
  floatRisePx: 40,        // 飘字上浮距离
  floatFadeMs: 600,       // 飘字渐隐
  comboResetMs: 1500,     // 连击中断判定
  comboMilestones: [10, 50, 100, 300, 500, 1000],
};
// -------- src/data/strings.ts --------
// 文案集中管理:偈语、连击文案、UI 文案
// 合规注意:禁用「随喜/供养/添灯油/祈福必应」等宗教献金暗示词
export const STRINGS = {
  title: '敲到成佛',
  subtitle: '一敲一世界,一念一菩提',
  guestTip: '离线禅修中,联网后自动同步',
  offlineTitle: '闭关收益',
  offlineBody: (hours: string, merit: number) => `闭关 ${hours} 小时\n静修功德 +${merit}`,
  offlineClaim: '领取',
  offlineDouble: '看视频翻倍',
  breakthroughTitle: '境界突破',
  breakthroughTap: '轻触继续',
  dailyTitle: '每日功课',
  dailyProgress: (taps: number, goal: number) => `今日功课 ${taps}/${goal}`,
  dailyClaim: '领取宝箱',
  dailyClaimed: '今日已领取',
  dailyGoalDone: '今日功课圆满',
  shareTitle: '我在《敲到成佛》修行,快来一起攒功德!',
  shareBtn: '分享 +100',
  navCultivate: '修行',
  navDaily: '功课',
  navRank: '排行',
  navSettings: '设置',
  adMeritDouble: '静心加成',
  adOfflineDouble: '离线翻倍',
  adLimitReached: '明日再来',
  adNotReady: '广告加载中,稍后再试',
  adGrace: '入静片刻,稍后再看',
  verseLocked: '敲满今日功课,一偈自现',
  ambienceRain: '雨声',
  ambienceBird: '鸟鸣',
  ornamentGot: '连续修行 7 日,获得挂饰「精进」',
  nextLevelPrefix: '距',
  nextLevelSuffix: '还差',
  meritUnit: '功德',
  confirmClear: '清除本地数据将丢失未同步进度,确定吗?',
  tapSoundLabel: '木鱼音色',
  tapSoundResonant: '腔体共鸣',
  tapSoundWooden: '木质敲击',
  tapSoundCrisp: '清脆实木',
```

```
  tapSoundThump: '咚咚木鱼',
};
// 连击里程碑文案
export const COMBO_TEXTS: Record<number, string> = {
  10: '心静如水',
  50: '法喜充满',
  100: '功德无量',
  300: '渐入佳境',
  500: '心无挂碍',
  1000: '明心见性',
};
// 每日一偈池
export const VERSES: string[] = [
  '菩提本无树,明镜亦非台',
  '一切有为法,如梦幻泡影',
  '应无所住,而生其心',
  '色即是空,空即是色',
  '一念放下,万般自在',
  '心若不动,风又奈何',
  '浮生若茶,苦后方知甘甜',
  '春有百花秋有月,夏有凉风冬有雪',
  '若无闲事挂心头,便是人间好时节',
  '千山鸟飞绝,万径人踪灭',
  '行到水穷处,坐看云起时',
  '本来无一物,何处惹尘埃',
  '竹影扫阶尘不动,月穿潭底水无痕',
  '不是风动,不是幡动,仁者心动',
  '万古长空,一朝风月',
  '溪声便是广长舌,山色无非清净身',
  '人生如逆旅,我亦是行人',
  '此心安处是吾乡',
  '问渠那得清如许,为有源头活水来',
  '不畏浮云遮望眼,自缘身在最高层',
];
// 新手引导
export const TUTORIAL_STEPS = [
  '轻触木鱼,开始修行',
  '每敲一下,功德 +1',
  '点击「修行」查看境界',
];
// -------- src/data/play.ts --------
// 听经、功法、入灭。广告和钥匙不参与这一层,听经随时可进。
export type NoteType = 'tap' | 'hold' | 'slide';
export type GradeId = 'supreme' | 'insight' | 'entry' | 'fail';
export interface Note {
  t: number;
  type: NoteType;
  pos: number;
  dur?: number;
  to?: number;
```

```
}
export interface ChartDef {
  id: string;
  name: string;
  unlockLevel: number;
  gongfaId: string;
  duration: number;
  gap: number;
  perfectMs: number;
  goodMs: number;
  cycle: NoteType[];
}
export interface GongfaDef {
  id: string;
  name: string;
  desc: string;
}
export const REBIRTH_MERIT = 1_000_000;
export const RELIC_FLAT = 0.5;
export const ENERGY_MAX = 100;
export const BODHI_COST = 10;
export const NOTE_CAP = 40;
export const GRADE_NAME: Record<GradeId, string> = {
  supreme: '无上正觉',
  insight: '明心见性',
  entry: '初窥门径',
  fail: '杂念纷飞',
};
export const CHARTS: ChartDef[] = [
  { id: 'xinjing', name: '心经', unlockLevel: 0, gongfaId: 'jingxin', duration: 22, gap: 0.9, perfectMs: 80, go
odMs: 140, cycle: ['tap', 'tap', 'tap', 'hold'] },
  { id: 'dabei', name: '大悲咒', unlockLevel: 1, gongfaId: 'cibei', duration: 24, gap: 0.64, perfectMs: 60, goo
dMs: 120, cycle: ['tap', 'tap', 'slide', 'tap'] },
  { id: 'jingang', name: '金刚经', unlockLevel: 2, gongfaId: 'bore', duration: 26, gap: 0.5, perfectMs: 50, goo
dMs: 110, cycle: ['tap', 'hold', 'tap', 'slide'] },
  { id: 'lengyan', name: '楞严咒', unlockLevel: 3, gongfaId: 'pomo', duration: 26, gap: 0.42, perfectMs: 48, go
odMs: 100, cycle: ['tap', 'slide', 'hold', 'tap'] },
  { id: 'huayan', name: '华严经', unlockLevel: 4, gongfaId: 'fajie', duration: 28, gap: 0.36, perfectMs: 45, go
odMs: 90, cycle: ['tap', 'slide', 'tap', 'hold'] },
];
export const GONGFA: GongfaDef[] = [
  { id: 'jingxin', name: '静心咒', desc: '敲击功德 +8%' },
  { id: 'cibei', name: '慈悲愿', desc: '连击不断的时间更长' },
  { id: 'bore', name: '般若力', desc: '听经判定更宽' },
  { id: 'pomo', name: '破魔印', desc: '驱散烦恼时功德更多' },
  { id: 'fajie', name: '法界观', desc: '离线禅修收益翻倍' },
  { id: 'dayuan', name: '大愿力', desc: '每日功课圆满时额外功德' },
];
export const MANTRAS = [
  { id: 'daming', name: '六字真言', unlockLevel: 1, desc: '能量满后一次 +80 功德' },
```

```
  { id: 'wangsheng', name: '净土莲华', unlockLevel: 3, desc: '能量满后 20 秒功德翻倍' },
];
export function slotCount(levelIndex: number): number {
  if (levelIndex >= 4) return 3;
  if (levelIndex >= 2) return 2;
  return 1;
}
// -------- src/data/practice.ts --------
// 木鱼年轮与今日修行。年轮不加成功德，短课奖励固定 +20。
export type LessonId = 'settle' | 'once' | 'listen' | 'copy';
export interface LessonDef {
  id: LessonId;
  name: string;
  goal: number;
}
export const RING_TAPS = [100, 500, 2000, 8000, 30000];
export const RING_SLOTS = 5;
export const NOTCH_CAP = 8;
export const SETTLE_GAP_MS = 400;
export const SETTLE_GOAL = 8;
export const ONCE_WAIT_MS = 1200;
export const LISTEN_MS = 20000;
export const COPY_DOTS = 6;
export const COPY_HIT_R = 28;
export const PRACTICE_REWARD = 20;
export const PRACTICE_STREAK_GLOW = 7;
export const LESSONS: LessonDef[] = [
  { id: 'settle', name: '安住', goal: SETTLE_GOAL },
  { id: 'once', name: '一击', goal: 1 },
  { id: 'listen', name: '静听', goal: 20 },
  { id: 'copy', name: '抄经', goal: COPY_DOTS },
];
export const COPY_CHARS = ['心', '安', '定', '静'];
export const VERSES: Record<LessonId, string[]> = {
  settle: [
    '慢一点，这一下才算数',
    '间隔里也有木头的声音',
    '不必赶，鱼还在',
    '手松一点，再敲',
    '八下安住，刚刚好',
    '太急的一下，就让它过去',
    '今天的节奏，由你放慢',
  ],
  once: [
    '一击之后，看涟漪走完',
    '停住，比再敲更难',
    '这一下已经够了',
    '等它散尽，再离开',
    '木鱼记得刚才那一声',
    '手放下，光还在散',
```

```
    '只留一下，其余都是余韵',
  ],
  listen: [
    '二十秒，只听不敲',
    '雨声也是功课',
    '手放下的时候，房间安静了',
    '不必填满每一秒',
    '静听本身就是一下',
    '木鱼先歇着',
    '回来的时候，它还在',
  ],
  copy: [
    '点对了，字才留下',
    '六下，沿着弧走',
    '点错了也不必重来',
    '顺着点，不要跳',
    '这一笔很短',
    '点完，字就留在鱼上',
    '慢慢点，鱼认得这个顺序',
  ],
};
// -------- src/utils/format.ts --------
// 格式化与日期工具
export function dateKeyOf(ts: number): string {
  // UTC+8 日期戳 YYYY-MM-DD
  const d = new Date(ts + 8 * 3600 * 1000);
  return d.toISOString().slice(0, 10);
}
export function todayKey(): string {
  return dateKeyOf(Date.now());
}
/** 大数字格式化:≥10000 用「万」 */
export function fmtNumber(n: number): string {
  if (n < 10000) return String(Math.floor(n));
  if (n < 100000000) {
    const w = n / 10000;
    return (w >= 100 ? Math.floor(w) : w.toFixed(1)) + '万';
  }
  const y = n / 100000000;
  return (y >= 100 ? Math.floor(y) : y.toFixed(1)) + '亿';
}
/** 时长格式化:1.5h / 25min */
export function fmtDuration(ms: number): string {
  const min = ms / 60000;
  if (min < 60) return `${Math.floor(min)}分钟`;
  const h = min / 60;
  return `${h.toFixed(1)}小时`;
}
export function clamp(v: number, min: number, max: number): number {
  return v < min ? min : v > max ? max : v;
```

```
}
export function lerp(a: number, b: number, t: number): number {
  return a + (b - a) * t;
}
// -------- src/utils/pool.ts --------
// 通用对象池:飘字/涟漪/粒子复用,避免频繁 GC
export class ObjectPool<T> {
  private free: T[] = [];
  private factory: () => T;
  private reset: (obj: T) => void;
  constructor(factory: () => T, reset: (obj: T) => void, prealloc = 8) {
    this.factory = factory;
    this.reset = reset;
    for (let i = 0; i < prealloc; i++) this.free.push(factory());
  }
  obtain(): T {
    return this.free.length ? this.free.pop()! : this.factory();
  }
  release(obj: T): void {
    this.reset(obj);
    this.free.push(obj);
  }
}
// -------- src/utils/dailyRules.ts --------
// 每日功课跨天规则。云函数 syncProfile / dailyClaim 必须保持同一口径。
import type { DailyState } from '../data/types';
export function rollDaily(daily: DailyState, today: string, yesterday: string): DailyState {
  if (daily.dateKey === today) return daily;
  // 只有「昨天领过」才保住连续天数；隔天或昨天没领，断签
  const streak = daily.dateKey === yesterday && daily.claimed ? daily.streak : 0;
  const practiceStreak = daily.dateKey === yesterday && daily.practiceDone ? daily.practiceStreak : 0;
  return {
    dateKey: today,
    taps: 0,
    claimed: false,
    streak,
    shareMeritClaimed: false,
    adWatch: {},
    practiceProgress: 0,
    practiceDone: false,
    practiceStreak,
  };
}
export function selfCheckDailyRules(): void {
  const yesterday = '2026-09-29';
  const today = '2026-09-30';
  const base = {
    dateKey: yesterday,
    taps: 100,
    claimed: true,
```

```
    streak: 3,
    shareMeritClaimed: true,
    adWatch: { offline_double: 1 },
    practiceProgress: 4,
    practiceDone: true,
    practiceStreak: 2,
  };
  const kept = rollDaily(base, today, yesterday);
  if (kept.streak !== 3 || kept.claimed || kept.taps !== 0 || kept.shareMeritClaimed) {
    throw new Error('yesterday claimed should keep streak and reset the day');
  }
  if (kept.practiceDone || kept.practiceProgress !== 0 || kept.practiceStreak !== 2) {
    throw new Error('yesterday practice should keep streak and reset progress');
  }
  const skipped = rollDaily({ ...base, dateKey: '2026-09-20' }, today, yesterday);
  if (skipped.streak !== 0 || skipped.practiceStreak !== 0) throw new Error('skipped day should break streak')
;
  const same = rollDaily({ ...base, dateKey: today, taps: 12, streak: 2, practiceProgress: 3 }, today, yesterd
ay);
  if (same.taps !== 12 || same.streak !== 2 || same.practiceProgress !== 3) {
    throw new Error('same day must not reset');
  }
}
// -------- src/utils/practiceRules.ts --------
import {
  COPY_CHARS,
  COPY_DOTS,
  LESSONS,
  LISTEN_MS,
  NOTCH_CAP,
  ONCE_WAIT_MS,
  RING_SLOTS,
  RING_TAPS,
  SETTLE_GAP_MS,
  SETTLE_GOAL,
  VERSES,
  type LessonDef,
  type LessonId,
} from '../data/practice';
export function ringCount(totalTaps: number): number {
  let n = 0;
  for (let i = 0; i < RING_TAPS.length; i++) {
    if (totalTaps >= RING_TAPS[i]) n += 1;
  }
  return n;
}
/** 第 index 圈(0 为最外圈)相对鱼半径的比例 */
export function ringRadiusRatio(index: number): number {
  if (RING_SLOTS <= 1) return 0.72;
  return 0.72 - (index * (0.72 - 0.28)) / (RING_SLOTS - 1);
```

```
}
export function ringPitch(totalTaps: number): number {
  return 1 - 0.02 * ringCount(totalTaps);
}
export function notchCount(cycle: number): number {
  const n = Math.floor(cycle);
  if (n <= 0) return 0;
  return n > NOTCH_CAP ? NOTCH_CAP : n;
}
/** 北京时间日期 YYYY-MM-DD 距 2026-01-01 的天数 */
export function dayIndexOf(dateKey: string): number {
  const parts = dateKey.split('-');
  if (parts.length !== 3) return 0;
  const y = Number(parts[0]);
  const m = Number(parts[1]);
  const d = Number(parts[2]);
  if (!y || !m || !d) return 0;
  const utc = Date.UTC(y, m - 1, d);
  const origin = Date.UTC(2026, 0, 1);
  return Math.floor((utc - origin) / 86400000);
}
export function lessonOf(dayIndex: number): LessonDef {
  const i = ((dayIndex % LESSONS.length) + LESSONS.length) % LESSONS.length;
  return LESSONS[i];
}
export function verseOf(dayIndex: number): string {
  const lesson = lessonOf(dayIndex);
  const lines = VERSES[lesson.id];
  const i = Math.floor(dayIndex / LESSONS.length) % lines.length;
  return lines[i];
}
export function copyCharOf(dayIndex: number): string {
  const i = ((Math.floor(dayIndex / LESSONS.length) % COPY_CHARS.length) + COPY_CHARS.length) % COPY_CHARS.len
gth;
  return COPY_CHARS[i];
}
/** 抄经点在鱼身局部坐标中的位置，单位是鱼半径 */
export function copyDotOffset(index: number): { x: number; y: number } {
  const t = (index + 0.5) / COPY_DOTS;
  return {
    x: -0.55 + t * 1.1,
    y: -0.08 - Math.sin(t * Math.PI) * 0.42,
  };
}
export function practiceLabel(lesson: LessonDef, progress: number, done: boolean): string {
  if (done) return '今日修行已圆满';
  if (progress <= 0) return `今日 · ${lesson.name}`;
  if (lesson.id === 'listen') {
    const sec = Math.min(lesson.goal, Math.floor(progress / 1000));
    return `今日 · ${lesson.name} ${sec}/${lesson.goal}`;
```

```
  }
  return `今日 · ${lesson.name} ${progress}/${lesson.goal}`;
}
export interface SettleResult {
  progress: number;
  countMerit: boolean;
  done: boolean;
  lastAt: number;
  hint: string;
}
export function applySettle(progress: number, now: number, lastAt: number): SettleResult {
  if (lastAt > 0 && now - lastAt < SETTLE_GAP_MS) {
    return { progress, countMerit: false, done: false, lastAt, hint: '太急' };
  }
  const next = progress + 1;
  return { progress: next, countMerit: true, done: next >= SETTLE_GOAL, lastAt: now, hint: '' };
}
export function applyOnceTap(): { progress: number; countMerit: boolean } {
  return { progress: 1, countMerit: true };
}
export function onceReady(waitMs: number): boolean {
  return waitMs >= ONCE_WAIT_MS;
}
export function listenReady(elapsedMs: number): boolean {
  return elapsedMs >= LISTEN_MS;
}
export interface CopyResult {
  progress: number;
  countMerit: boolean;
  done: boolean;
  shake: boolean;
}
export function applyCopy(progress: number, hitIndex: number): CopyResult {
  if (hitIndex === progress) {
    const next = progress + 1;
    return { progress: next, countMerit: true, done: next >= COPY_DOTS, shake: false };
  }
  return { progress, countMerit: false, done: false, shake: true };
}
export function lessonIdOf(dayIndex: number): LessonId {
  return lessonOf(dayIndex).id;
}
// -------- game.js --------
// 小游戏入口:加载 TypeScript 编译产物
// 开发流程: npm run watch(tsc 监听编译 src → dist),微信开发者工具打开本目录
require('./dist/main.js');
// -------- open-data/index.js --------
// 开放数据域:好友功德榜渲染
// 运行在独立上下文,只能访问 wx.getFriendCloudStorage / wx.getUserCloudStorage
// 主域通过 postMessage 通信(传入渲染区域宽高与 dpr,避免画布被拉伸变形)
```

```
let WIDTH = 375;
let HEIGHT = 500;
let dpr = 1;
let running = false;
let renderTimer = null;
wx.onMessage((data) => {
  if (data.type === 'renderRank') {
    dpr = data.dpr || 1;
    WIDTH = data.width || 375;
    HEIGHT = data.height || 500;
    // 按传入尺寸 + dpr 设置画布物理分辨率,主域按等比绘制不再拉伸
    const canvas = wx.getSharedCanvas();
    canvas.width = Math.floor(WIDTH * dpr);
    canvas.height = Math.floor(HEIGHT * dpr);
    startRender();
  } else if (data.type === 'stop') {
    stopRender();
  }
});
function startRender() {
  if (running) return;
  running = true;
  fetchAndDraw();
  // 每 30s 刷新一次
  renderTimer = setInterval(fetchAndDraw, 30000);
}
function stopRender() {
  running = false;
  if (renderTimer) {
    clearInterval(renderTimer);
    renderTimer = null;
  }
}
function fetchAndDraw() {
  wx.getFriendCloudStorage({
    keyList: ['merit', 'level'],
    success: (res) => {
      const list = (res.data || [])
        .map((u) => ({
          nickname: u.nickname || '同修',
          avatarUrl: u.avatarUrl || '',
          merit: parseInt(getKV(u.KVDataList, 'merit') || '0', 10),
          level: getKV(u.KVDataList, 'level') || '凡夫',
        }))
        .sort((a, b) => b.merit - a.merit);
      drawRank(list);
    },
    fail: (e) => {
      console.warn('[open-data] 获取好友数据失败:', e);
      drawRank([]);
```

```
    },
  });
}
function getKV(kvList, key) {
  if (!kvList) return null;
  const item = kvList.find((kv) => kv.key === key);
  return item ? item.value : null;
}
function fillCenter(ctx, text, y) {
  // 开放域里 textAlign=center 不可靠，用测量宽度自己居中
  ctx.textAlign = 'left';
  ctx.textBaseline = 'middle';
  const tw = ctx.measureText(text).width;
  ctx.fillText(text, Math.max(0, (WIDTH - tw) / 2), y);
}
function drawRank(list) {
  const canvas = wx.getSharedCanvas();
  const ctx = canvas.getContext('2d');
  // 逻辑坐标 = 物理像素 / dpr
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  const w = WIDTH;
  // 清空
  ctx.clearRect(0, 0, WIDTH, HEIGHT);
  if (!list.length) {
    ctx.fillStyle = '#9A8F74';
    ctx.font = '14px sans-serif';
    fillCenter(ctx, '暂无好友数据', HEIGHT / 2 - 12);
    fillCenter(ctx, '分享给好友一起修行吧', HEIGHT / 2 + 12);
    return;
  }
  const rowH = 52;
  const startY = 8;
  list.slice(0, 20).forEach((user, i) => {
    const y = startY + i * rowH;
    // 背景条
    ctx.fillStyle = i < 3 ? 'rgba(232,184,75,0.1)' : 'rgba(255,255,255,0.03)';
    ctx.fillRect(0, y, w, rowH - 4);
    // 排名
    ctx.fillStyle = i < 3 ? '#E8B84B' : '#9A8F74';
    ctx.font = i < 3 ? 'bold 16px sans-serif' : '14px sans-serif';
    ctx.textAlign = 'left';
    ctx.textBaseline = 'middle';
    ctx.fillText(String(i + 1), 12, y + rowH / 2 - 2);
    // 昵称
    ctx.fillStyle = '#F5EDD8';
    ctx.font = '15px sans-serif';
    ctx.fillText(user.nickname, 44, y + rowH / 2 - 2);
    // 境界
    ctx.fillStyle = '#9A8F74';
    ctx.font = '12px sans-serif';
```

```
    ctx.textAlign = 'right';
    ctx.fillText(user.level, w - 100, y + rowH / 2 - 2);
    // 功德
    ctx.fillStyle = '#E8B84B';
    ctx.font = 'bold 14px sans-serif';
    ctx.fillText(String(user.merit), w - 12, y + rowH / 2 - 2);
    ctx.textAlign = 'left';
  });
}
// -------- cloud/login/index.js --------
// 登录云函数:获取 openid、创建用户与存档
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const DEFAULT_PROFILE = {
  merit: 0,
  totalTaps: 0,
  skinId: 'classic_wood',
  sceneId: 'temple',
  bgmId: 'none',
  soundOn: true,
  vibrateOn: true,
  tapSound: 'resonant',
  inventory: { skins: ['classic_wood'], scenes: ['temple'], bgms: [] },
  daily: { dateKey: '', taps: 0, claimed: false, streak: 0, shareMeritClaimed: false, adWatch: {}, practicePro
gress: 0, practiceDone: false, practiceStreak: 0 },
  lastSeenAt: null,
  lastSyncAt: null,
  offlineClaimedAt: 0,
  cycle: 0,
  relics: 0,
  levelPeak: 0,
  extra: {},
};
exports.main = async () => {
  const { OPENID } = cloud.getWXContext();
  const now = new Date();
  const users = db.collection('users');
  const profiles = db.collection('profiles');
  const existing = await users.where({ _openid: OPENID }).limit(1).get();
  const isNew = existing.data.length === 0;
  if (isNew) {
    await users.add({
      data: { _openid: OPENID, nickname: '', avatarUrl: '', createdAt: now, updatedAt: now },
    });
    await profiles.add({
      data: { _openid: OPENID, ...DEFAULT_PROFILE, lastSeenAt: now, lastSyncAt: now },
    });
  }
  const profileRes = await profiles.where({ _openid: OPENID }).limit(1).get();
```

```
  return {
    openid: OPENID,
    isNew,
    profile: profileRes.data[0] || null,
    serverTime: now.getTime(),
  };
};
// -------- cloud/syncProfile/index.js --------
// 存档同步云函数:字段级合并 + 防刷校验
// 防刷:敲击频率上限 15 次/秒 × 时间窗口;异常增量截断;
//       meritDelta 上限(敲击 + 离线/宝箱/分享奖励);库存只接受已解锁项
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
const MAX_TAPS_PER_SECOND = 15;
// 单次上报允许的最大非敲击奖励(离线翻倍 12h×300×系数≈1.4 万,留余量)
const MAX_BONUS_MERIT = 20000;
// 敲击功德系数上限(皮肤最高 +10%,留余量)
const MAX_MERIT_PER_TAP = 2;
// 与客户端 configs.ts 保持一致:境界门槛 / 皮肤 / 场景 / BGM 解锁等级
const LEVEL_MERITS = [0, 1000, 10000, 100000, 1000000, 10000000];
const SKIN_UNLOCKS = { classic_wood: 0, sandalwood: 1, jade: 2 };
const EXTRA_SKINS = { mist: true, sunbird: true };
const SCENE_UNLOCKS = { temple: 0, bamboo: 3, ridge: 4 };
const BGM_UNLOCKS = { xinjing: 1, dabeizhou: 2, jingangjing: 3 };
function todayKey() {
  const d = new Date(Date.now() + 8 * 3600 * 1000);
  return d.toISOString().slice(0, 10);
}
function yesterdayKey() {
  const d = new Date(Date.now() + 8 * 3600 * 1000 - 86400000);
  return d.toISOString().slice(0, 10);
}
function practiceOf(daily, sameDay, yesterday) {
  if (sameDay) {
    return {
      practiceProgress: daily.practiceProgress || 0,
      practiceDone: !!daily.practiceDone,
      practiceStreak: daily.practiceStreak || 0,
    };
  }
  return {
    practiceProgress: 0,
    practiceDone: false,
    practiceStreak: daily.dateKey === yesterday && daily.practiceDone ? (daily.practiceStreak || 0) : 0,
  };
}
/** 同一天取进度较大的一侧，避免云同步把今日修行盖掉 */
function takePractice(rolled, client) {
```

```
  if (!client || client.dateKey !== rolled.dateKey) return rolled;
  const incoming = Math.max(0, Math.min(20000, Math.floor(Number(client.practiceProgress) || 0)));
  rolled.practiceProgress = Math.max(rolled.practiceProgress || 0, incoming);
  rolled.practiceDone = !!(rolled.practiceDone || client.practiceDone);
  rolled.practiceStreak = Math.max(
    rolled.practiceStreak || 0,
    Math.max(0, Math.min(9999, Math.floor(Number(client.practiceStreak) || 0)))
  );
  return rolled;
}
/** 与客户端 dailyRules.rollDaily 同一口径;tapsToAdd 只加在当天 */
function rollServerDaily(prev, today, tapsToAdd) {
  const daily = prev || {};
  if (daily.dateKey === today) {
    return {
      dateKey: today,
      taps: (daily.taps || 0) + tapsToAdd,
      claimed: !!daily.claimed,
      streak: daily.streak || 0,
      shareMeritClaimed: !!daily.shareMeritClaimed,
      adWatch: daily.adWatch || {},
      ...practiceOf(daily, true, ''),
    };
  }
  const yesterday = yesterdayKey();
  return {
    dateKey: today,
    taps: tapsToAdd,
    claimed: false,
    streak: daily.dateKey === yesterday && daily.claimed ? (daily.streak || 0) : 0,
    shareMeritClaimed: false,
    adWatch: {},
    ...practiceOf(daily, false, yesterday),
  };
}
function levelIndexOf(merit) {
  let idx = 0;
  for (let i = 0; i < LEVEL_MERITS.length; i++) {
    if (merit >= LEVEL_MERITS[i]) idx = i;
  }
  return idx;
}
/** 过滤未解锁项(服务端权威,防伪造库存) */
function filterUnlocked(list, unlockMap, levelIdx) {
  if (!Array.isArray(list)) return [];
  return list.filter((id) => (unlockMap[id] !== undefined && unlockMap[id] <= levelIdx) || EXTRA_SKINS[id]);
}
const GONGFA_IDS = ['jingxin', 'cibei', 'bore', 'pomo', 'fajie', 'dayuan'];
const MANTRA_IDS = ['daming', 'wangsheng', ''];
function sanitizePlay(clientPlay, doc) {
```

```
  const src = clientPlay || {};
  const learned = (Array.isArray(src.learned) ? src.learned : []).filter((id) => GONGFA_IDS.includes(id));
  const mastered = (Array.isArray(src.mastered) ? src.mastered : []).filter((id) => learned.includes(id));
  const slots = (Array.isArray(src.slots) ? src.slots : []).filter((id) => learned.includes(id)).slice(0, 3);
  const levelPeak = Math.max(doc.levelPeak || 0, Math.max(0, Math.min(5, Math.floor(src.levelPeak || 0))));
  return {
    learned,
    mastered,
    slots,
    mantra: MANTRA_IDS.includes(src.mantra) ? src.mantra : '',
    energy: Math.max(0, Math.min(100, Math.floor(src.energy || 0))),
    bodhi: Math.max(0, Math.min(999, Math.floor(src.bodhi || 0))),
    grades: src.grades && typeof src.grades === 'object' ? src.grades : {},
    levelPeak,
    cycle: doc.cycle || 0,
    relics: doc.relics || 0,
    pendingRebirth: false,
  };
}
exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext();
  const { tapsDelta = 0, meritDelta = 0, profile: clientProfile } = event;
  const now = new Date();
  const today = todayKey();
  const profiles = db.collection('profiles');
  const res = await profiles.where({ _openid: OPENID }).limit(1).get();
  if (res.data.length === 0) return { ok: false, err: 'no_profile' };
  const doc = res.data[0];
  if (event.rebirth) {
    if ((doc.merit || 0) < 1000000) return { ok: false, err: 'not_enough', merit: doc.merit || 0 };
    const cycle = (doc.cycle || 0) + 1;
    const relics = (doc.relics || 0) + 1;
    await profiles.doc(doc._id).update({
      data: { merit: 0, cycle, relics, lastSyncAt: now, lastSeenAt: now },
    });
    return { ok: true, cycle, relics, merit: 0 };
  }
  const lastSync = doc.lastSyncAt ? new Date(doc.lastSyncAt).getTime() : now.getTime();
  const elapsedSec = Math.max(1, (now.getTime() - lastSync) / 1000);
  // 防刷 1:时间窗口内允许的最大敲击数(宽松 2 倍,避免误伤)
  const maxAllowed = Math.ceil(elapsedSec * MAX_TAPS_PER_SECOND * 2);
  let taps = Math.max(0, Math.floor(tapsDelta));
  let truncated = false;
  if (taps > maxAllowed) {
    taps = maxAllowed;
    truncated = true;
    console.warn(`[syncProfile] 异常增量截断: ${tapsDelta} → ${taps} (openid=${OPENID})`);
  }
  // 防刷 2:功德增量上限(敲击部分 ≤ taps×系数 + 单次奖励余量)
  let merit = Math.max(0, Math.floor(meritDelta));
```

```
  const meritCap = taps * MAX_MERIT_PER_TAP + MAX_BONUS_MERIT;
  if (merit > meritCap) {
    console.warn(`[syncProfile] 功德增量截断: ${merit} → ${meritCap} (openid=${OPENID})`);
    merit = meritCap;
    truncated = true;
  }
  // 用更新后功德计算境界,再决定库存/装备合法性
  const nextMerit = (doc.merit || 0) + merit;
  const levelIdx = levelIndexOf(nextMerit);
  const updateData = {
    totalTaps: _.inc(taps),
    merit: _.inc(merit),
    lastSyncAt: now,
    lastSeenAt: now,
  };
  // 字段级合并:客户端带来的非数值字段,仅接受已解锁项
  if (clientProfile) {
    if (clientProfile.skinId && SKIN_UNLOCKS[clientProfile.skinId] <= levelIdx) {
      updateData.skinId = clientProfile.skinId;
    }
    if (clientProfile.sceneId && SCENE_UNLOCKS[clientProfile.sceneId] <= levelIdx) {
      updateData.sceneId = clientProfile.sceneId;
    }
    if (clientProfile.bgmId && (BGM_UNLOCKS[clientProfile.bgmId] !== undefined ? BGM_UNLOCKS[clientProfile.bgm
Id] <= levelIdx : false)) {
      updateData.bgmId = clientProfile.bgmId;
    }
    if (typeof clientProfile.soundOn === 'boolean') updateData.soundOn = clientProfile.soundOn;
    if (typeof clientProfile.vibrateOn === 'boolean') updateData.vibrateOn = clientProfile.vibrateOn;
    if (['resonant', 'wooden', 'crisp', 'thump'].includes(clientProfile.tapSound)) updateData.tapSound = clien
tProfile.tapSound;
    if (clientProfile.inventory) {
      updateData.inventory = {
        skins: Array.from(new Set([
          ...(doc.inventory && doc.inventory.skins ? doc.inventory.skins : []),
          ...filterUnlocked(clientProfile.inventory.skins, SKIN_UNLOCKS, levelIdx),
        ])),
        scenes: Array.from(new Set([
          ...(doc.inventory && doc.inventory.scenes ? doc.inventory.scenes : []),
          ...filterUnlocked(clientProfile.inventory.scenes, SCENE_UNLOCKS, levelIdx),
        ])),
        bgms: Array.from(new Set([
          ...(doc.inventory && doc.inventory.bgms ? doc.inventory.bgms : []),
          ...filterUnlocked(clientProfile.inventory.bgms, BGM_UNLOCKS, levelIdx),
        ])),
      };
    }
  }
  // 每日功课:跨天必须清掉昨天的领取标记,否则第二天永远领不到
  updateData.daily = takePractice(
```

```
    rollServerDaily(doc.daily, today, taps),
    clientProfile && clientProfile.practice
  );
  const play = sanitizePlay(clientProfile && clientProfile.play, doc);
  updateData.extraPlay = play;
  updateData.levelPeak = play.levelPeak;
  await profiles.doc(doc._id).update({ data: updateData });
  return { ok: true, truncated, serverTime: now.getTime() };
};
// -------- cloud/offlineCalc/index.js --------
// 离线收益权威结算:以云端 lastSeenAt 为准,客户端仅展示预估值
// 防刷:翻倍(doubled)必须当日广告已观看,服务端记录次数,客户端不可直接传 true
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
const TAPS_PER_HOUR = 300;
const CAP_HOURS = 12;
const LEVEL_COEFF = 0.2; // 每阶 +20%
const OFFLINE_UNLOCK_LEVEL = 3; // 罗汉解锁
// 境界表(与客户端 configs.ts 保持一致)
const LEVELS = [0, 1000, 10000, 100000, 1000000, 10000000];
function todayKey() {
  const d = new Date(Date.now() + 8 * 3600 * 1000);
  return d.toISOString().slice(0, 10);
}
function levelIndexOf(merit) {
  let idx = 0;
  for (let i = 0; i < LEVELS.length; i++) {
    if (merit >= LEVELS[i]) idx = i;
  }
  return idx;
}
exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext();
  const { doubled = false } = event;
  const now = new Date();
  const today = todayKey();
  const profiles = db.collection('profiles');
  const res = await profiles.where({ _openid: OPENID }).limit(1).get();
  if (res.data.length === 0) return { ok: false, err: 'no_profile' };
  const doc = res.data[0];
  const lastSeen = doc.lastSeenAt ? new Date(doc.lastSeenAt).getTime() : now.getTime();
  const hours = Math.min((now.getTime() - lastSeen) / 3600000, CAP_HOURS);
  // 罗汉(境界 3)解锁离线禅修
  const levelIdx = Math.max(levelIndexOf(doc.merit || 0), doc.levelPeak || 0);
  if (levelIdx < OFFLINE_UNLOCK_LEVEL) return { ok: false, err: 'not_unlocked', merit: 0 };
  if (hours < 0.05) return { ok: true, merit: 0, hours: 0 }; // 不足 3 分钟
  // 翻倍必须当日广告已观看(每日 1 次),服务端校验
  const daily = doc.daily || {};
```

```
  if (doubled) {
    const watched = daily.dateKey === today && (daily.adWatch && daily.adWatch.offline_double || 0) > 0;
    if (!watched) {
      return { ok: false, err: 'doubled_not_authorized', merit: 0 };
    }
  }
  const slots = (doc.extraPlay && doc.extraPlay.slots) || [];
  const mastered = (doc.extraPlay && doc.extraPlay.mastered) || [];
  let gongfaMul = 1;
  if (slots.includes('fajie')) gongfaMul = mastered.includes('fajie') ? 2.4 : 2;
  const coeff = (1 + levelIdx * LEVEL_COEFF) * gongfaMul;
  let merit = Math.floor(TAPS_PER_HOUR * hours * coeff);
  if (doubled) merit *= 2;
  const updateData = {
    merit: _.inc(merit),
    offlineClaimedAt: now,
    lastSeenAt: now,
  };
  await profiles.doc(doc._id).update({ data: updateData });
  return { ok: true, merit, hours: Math.round(hours * 100) / 100 };
};
// -------- cloud/dailyClaim/index.js --------
// 每日功课领奖 + 分享得功德(每日一次,防刷)
// 防刷:功课 taps 只能由 syncProfile 累计(服务端权威),客户端不可直接传值绕过
const cloud = require('wx-server-sdk');
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV });
const db = cloud.database();
const _ = db.command;
const DAILY_GOAL = 100;
const REWARDS = [66, 88, 108, 168];
const SHARE_REWARD = 100;
const AD_LIMITS = { merit_double: 3, offline_double: 1, skin_trial: 1 };
const AD_DAILY_TOTAL = 6;
function todayKey() {
  const d = new Date(Date.now() + 8 * 3600 * 1000);
  return d.toISOString().slice(0, 10);
}
function yesterdayKey() {
  const d = new Date(Date.now() + 8 * 3600 * 1000 - 86400000);
  return d.toISOString().slice(0, 10);
}
function practiceOf(daily, sameDay, yesterday) {
  if (sameDay) {
    return {
      practiceProgress: daily.practiceProgress || 0,
      practiceDone: !!daily.practiceDone,
      practiceStreak: daily.practiceStreak || 0,
    };
  }
  return {
```

```
    practiceProgress: 0,
    practiceDone: false,
    practiceStreak: daily.dateKey === yesterday && daily.practiceDone ? (daily.practiceStreak || 0) : 0,
  };
}
function rollServerDaily(prev, today, tapsToAdd) {
  const daily = prev || {};
  if (daily.dateKey === today) {
    return {
      dateKey: today,
      taps: (daily.taps || 0) + tapsToAdd,
      claimed: !!daily.claimed,
      streak: daily.streak || 0,
      shareMeritClaimed: !!daily.shareMeritClaimed,
      adWatch: daily.adWatch || {},
      ...practiceOf(daily, true, ''),
    };
  }
  const yesterday = yesterdayKey();
  return {
    dateKey: today,
    taps: tapsToAdd,
    claimed: false,
    streak: daily.dateKey === yesterday && daily.claimed ? (daily.streak || 0) : 0,
    shareMeritClaimed: false,
    adWatch: {},
    ...practiceOf(daily, false, yesterday),
  };
}
exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext();
  const { type = 'quest' } = event; // 'quest' | 'share'
  const today = todayKey();
  const profiles = db.collection('profiles');
  const res = await profiles.where({ _openid: OPENID }).limit(1).get();
  if (res.data.length === 0) return { ok: false, err: 'no_profile' };
  const doc = res.data[0];
  if (type === 'ad') {
    const tag = event.tag;
    if (!AD_LIMITS[tag]) return { ok: false, err: 'bad_tag' };
    const daily = rollServerDaily(doc.daily, today, 0);
    const watch = { ...(daily.adWatch || {}) };
    const total = Object.keys(watch).reduce((sum, key) => sum + (watch[key] || 0), 0);
    if (total >= AD_DAILY_TOTAL || (watch[tag] || 0) >= AD_LIMITS[tag]) {
      return { ok: false, err: 'limit' };
    }
    watch[tag] = (watch[tag] || 0) + 1;
    daily.adWatch = watch;
    await profiles.doc(doc._id).update({ data: { daily } });
    return { ok: true };
```

```
  }
  // 分享得功德。先按当天滚动,避免昨天的已领标记带到今天
  if (type === 'share') {
    const daily = rollServerDaily(doc.daily, today, 0);
    if (daily.shareMeritClaimed) return { ok: false, err: 'already_claimed' };
    daily.shareMeritClaimed = true;
    await profiles.doc(doc._id).update({
      data: {
        daily,
        merit: _.inc(SHARE_REWARD),
      },
    });
    return { ok: true, reward: SHARE_REWARD };
  }
  // 每日功课宝箱:taps 以 syncProfile 写入的当天计数为准
  const daily = doc.daily || {};
  if (daily.dateKey === today && daily.claimed) {
    return { ok: false, err: 'already_claimed' };
  }
  if (daily.dateKey !== today || (daily.taps || 0) < DAILY_GOAL) {
    return { ok: false, err: 'not_enough_taps' };
  }
  // 连续天数已在跨天时裁过,这里只 +1
  const streak = (daily.streak || 0) + 1;
  const reward = REWARDS[Math.floor(Math.random() * REWARDS.length)];
  await profiles.doc(doc._id).update({
    data: {
      daily: {
        dateKey: today,
        taps: daily.taps || 0,
        claimed: true,
        streak,
        shareMeritClaimed: !!daily.shareMeritClaimed,
        adWatch: daily.adWatch || {},
        practiceProgress: daily.practiceProgress || 0,
        practiceDone: !!daily.practiceDone,
        practiceStreak: daily.practiceStreak || 0,
      },
      merit: _.inc(reward),
    },
  });
  return { ok: true, reward, streak };
};
```